

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN**

o0o



BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO 2-3-25(N01)

**ĐỀ TÀI: ỨNG DỤNG HỌC TĂNG CƯỜNG TRONG TRÒ CHƠI
PACMAN**

Đặng Anh Tuyên

MSSV: 23010912

Lớp: K17-KHMT(AI&KHDL)_1

Nguyễn Thị Khánh Linh

MSSV: 23016112

Lớp: K17-KHMT(AI&KHDL)_1

Lý Thành Đạt

MSSV: 23010879

Lớp: K17-KHMT(AI&KHDL)_1

GVHD: TS. Trần Đình Tân

Hà Nội, Tháng 8 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN**

o0o

Đặng Anh Tuyền

MSSV: 23010912

Lớp: K17-KHMT(AI&KHDL)_1

Nguyễn Thị Khánh Linh

MSSV: 23016112

Lớp: K17-KHMT(AI&KHDL)_1

Lý Thành Đạt

MSSV: 23010879

Lớp: K17-KHMT(AI&KHDL)_1

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO 2-3-25(N01)

**ỨNG DỤNG HỌC TĂNG CƯỜNG
TRONG TRÒ CHƠI PACMAN**

GVHD: TS. Trần Đình Tân

Hà Nội, Tháng 8 năm 2026

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	Vai trò	Nội dung công việc	Tỷ lệ
1	Đặng Anh Tuyền	Trưởng nhóm	– Lập kế hoạch, phân công và theo dõi tiến độ chung. – Xây dựng, tích hợp môi trường Mini Pacman. – Nghiên cứu nguyên lý hoạt động của thuật toán DDQN. – Cài đặt và cấu hình Q-Learning, DQN, Double DQN. – Tổ chức quy trình huấn luyện và đánh giá chín cấu hình. – Tổng hợp, đối chiếu kết quả thực nghiệm. – Biên tập, chuẩn hóa và hoàn thiện báo cáo. – Xây dựng, tổng hợp và hoàn thiện slide thuyết trình.	40%
2	Nguyễn Thị Khánh Linh	Thành viên	– Tìm hiểu cơ sở lý thuyết về học tăng cường. – Nghiên cứu nguyên lý và công thức cập nhật Q-Learning. – Viết nội dung lý thuyết, thu thập tài liệu tham khảo về Q-Learning. – Phân tích và trình bày kết quả Q-Learning từ số liệu do trưởng nhóm cung cấp. – Rà soát nội dung, chỉnh tả và hình thức báo cáo. – Chuẩn bị slide và thuyết trình phần Q-Learning.	30%
3	Lý Thành Đạt	Thành viên	– Tìm hiểu cơ sở lý thuyết về mạng nơ-ron sâu. – Nghiên cứu nguyên lý hoạt động của thuật toán DQN. – Viết nội dung lý thuyết, chuẩn bị sơ đồ minh họa kiến trúc DQN. – Phân tích và trình bày kết quả DQN từ số liệu do trưởng nhóm cung cấp. – Rà soát nội dung, chỉnh tả và hình thức báo cáo. – Chuẩn bị slide và thuyết trình phần DQN.	30%

Repository dự án: github.com/tuyenda2011/DRL-Pacman

Mục lục

1	GIỚI THIỆU ĐỀ TÀI	6
1.1	Bối cảnh và lý do lựa chọn	6
1.2	Mục tiêu nghiên cứu	7
1.3	Đối tượng và phạm vi nghiên cứu	7
1.4	Đóng góp của đề tài	8
2	CƠ SỞ LÝ THUYẾT	9
2.1	Tổng quan về học tăng cường	9
2.1.1	Quá trình quyết định Markov	9
2.1.2	Phần thưởng tích lũy và hàm giá trị	10
2.1.3	Khám phá và khai thác	11
2.2	Q-Learning	12
2.3	Deep Q-Network	13
2.3.1	Mạng Q	14
2.3.2	Experience replay	14
2.3.3	Target network và loss	15
2.4	Double DQN	16
2.5	So sánh Q-Learning, DQN và Double DQN	18
3	XÂY DỰNG HỆ THỐNG VÀ THIẾT KẾ THỰC NGHIỆM	19
3.1	Tổng quan ứng dụng học tăng cường trong trò chơi Pacman	19
3.2	Môi trường Mini Pacman	20
3.2.1	Bản đồ và các thành phần trong môi trường	20
3.2.2	Luật hoạt động và điều kiện kết thúc	21
3.3	Mô hình hóa bài toán học tăng cường	23
3.3.1	Không gian và biểu diễn trạng thái	23
3.3.2	Không gian hành động	25
3.3.3	Thiết kế phần thưởng	25
3.4	Cài đặt các thuật toán	26
3.4.1	Q-Learning	26
3.4.2	DQN	27
3.4.3	Double DQN	28

3.5	Thiết kế thực nghiệm	28
3.5.1	Cấu hình và siêu tham số	29
3.5.2	Quy trình huấn luyện và đánh giá	30
3.5.3	Tiêu chí đánh giá	31
4	KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN	34
4.1	Kết quả đánh giá cuối	34
4.2	Phân tích quá trình huấn luyện	35
4.2.1	Reward	36
4.2.2	Completion rate	38
4.2.3	Win rate	40
4.2.4	Steps	42
4.2.5	Loss	44
4.3	Ảnh hưởng của learning rate	45
4.4	Tổng hợp kết quả thực nghiệm	46
4.5	Hạn chế của thực nghiệm	47
4.6	Hướng dẫn chạy demo	48
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	50
5.1	Kết luận	50
5.2	Hướng phát triển	51
	TÀI LIỆU THAM KHẢO	53

Danh mục hình ảnh

2.1	Tương tác giữa tác tử và môi trường trong MDP [1].	9
2.2	Quy trình lựa chọn hành động và cập nhật Q-table của Q-Learning [1,2].	13
2.3	Luồng huấn luyện DQN với experience replay và target network [3,4].	16
2.4	Cơ chế lựa chọn và đánh giá hành động khi tính target Double DQN [5].	17
3.1	Kiến trúc tổng thể của ứng dụng học tăng cường trong trò chơi Pacman.	19
3.2	Bản đồ medium của môi trường Mini Pacman trong một lượt chạy demo.	21
3.3	Các trạng thái hoạt động và quá trình chuyển đổi của ma trong môi trường Mini Pacman.	22
3.4	Hai cách biểu diễn state được sử dụng cho Q-Learning và các phương pháp deep Q-learning.	24
3.5	Kiến trúc Q-network và vai trò của online network, target network trong hai phương pháp.	27
3.6	Quy trình training, lưu checkpoint, evaluation và lưu mô hình cuối.	30
4.1	Reward trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.	36
4.2	Completion rate trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.	38
4.3	Tỷ lệ thắng tích lũy trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.	40
4.4	Số bước trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.	42

4.5	Smooth L1 Loss trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.	44
4.6	Danh sách lựa chọn final model khi chạy demo.	48
4.7	Hai trạng thái kết thúc có thể quan sát trong giao diện demo. . . .	49

Danh mục bảng

2.1	So sánh Q-Learning, DQN và Double DQN về mặt nguyên lý. . .	18
3.1	Các thành phần của bản đồ medium [7].	20
3.2	Mục tiêu di chuyển của ba ma trong chế độ chase và scatter [7]. .	22
3.3	Cấu trúc vector state của DQN và Double DQN [7].	24
3.4	Các thành phần reward của môi trường Mini Pacman [7].	26
3.5	Các tham số chung của chín cấu hình thực nghiệm [7].	29
3.6	Các tham số riêng của Q-Learning, DQN và Double DQN [7]. . .	30
3.7	Các tiêu chí được sử dụng để đánh giá kết quả thực nghiệm. . . .	33
4.1	Kết quả đánh giá hiệu năng cuối cùng của các thuật toán tại episode 20.000 [7].	34
4.2	Reward trung bình tại các mốc đánh giá.	37
4.3	Completion rate trung bình tại các mốc đánh giá.	39
4.4	Win rate tại các mốc đánh giá.	41
4.5	Số bước trung bình tại các mốc đánh giá.	43
4.6	Loss trung bình trong episode tại các mốc training.	45
4.7	Xu hướng kết quả khi learning rate tăng từ 0,0001 lên 0,001 [7]. .	45
4.8	Đối chiếu ba cấu hình đại diện tại episode 20.000 [7].	46

1 GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và lý do lựa chọn

Trong trí tuệ nhân tạo (Artificial Intelligence – AI), ra quyết định tuần tự là lớp bài toán trong đó hệ thống phải lựa chọn hành động dựa trên trạng thái hiện tại, đồng thời xem xét ảnh hưởng của quyết định đó đối với các trạng thái và kết quả về sau. Học tăng cường (Reinforcement Learning – RL) là một hướng tiếp cận cho lớp bài toán này. Thay vì được cung cấp sẵn hành động đúng cho từng tình huống, tác tử (agent) tương tác với môi trường, nhận phần thưởng và điều chỉnh cách lựa chọn hành động nhằm tối đa hóa phần thưởng tích lũy [1].

Trên cơ sở đó, nhóm lựa chọn Pacman làm môi trường xây dựng và đánh giá các thuật toán học tăng cường [6]. Luật chơi trực quan nhưng vẫn thể hiện rõ đặc điểm của một bài toán ra quyết định tuần tự. Tại mỗi bước, tác tử phải lựa chọn hướng di chuyển dựa trên vị trí hiện tại, các vật phẩm còn lại và sự xuất hiện của các mối nguy hiểm trong mê cung. Thay vì sử dụng nguyên trạng một môi trường có sẵn, nhóm trực tiếp thiết kế và cài đặt Mini Pacman với bản đồ, không gian trạng thái, không gian hành động, cơ chế phần thưởng và điều kiện kết thúc riêng cho đề tài. Nhờ đó, nhóm có thể kiểm soát các yếu tố ảnh hưởng đến quá trình học và theo dõi trực tiếp hành vi của tác tử.

Trên môi trường này, nhóm cài đặt Q-Learning, DQN và Double DQN trong cùng một ứng dụng. Q-Learning biểu diễn giá trị của các cặp trạng thái–hành động bằng Q-table [2]. DQN sử dụng mạng nơ-ron để xấp xỉ hàm giá trị hành động, kết hợp replay buffer và target network trong quá trình huấn luyện [3,4]. Double DQN thay đổi cách xây dựng giá trị mục tiêu nhằm giảm xu hướng đánh giá quá cao (overestimation) của DQN [5]. Việc triển khai ba thuật toán giúp nhóm khảo sát sự khác biệt giữa phương pháp dạng bảng, phương pháp xấp xỉ bằng mạng nơ-ron và một cải tiến trực tiếp của DQN. Trong quá trình thực hiện đề tài, mã nguồn và các cấu hình được nhóm phát triển, quản lý trong repository DRL-Pacman trên GitHub. Kết quả thực nghiệm cũng được lưu trữ và công bố tại đây [7].

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng môi trường Mini Pacman, cài đặt và so sánh Q-Learning, DQN và Double DQN trong bài toán điều khiển tác tử chơi Pacman. Qua đó, nhóm xem xét mối liên hệ giữa nguyên lý của từng thuật toán, cách triển khai trong ứng dụng và kết quả quan sát được từ thực nghiệm.

Các mục tiêu cụ thể gồm:

- Xây dựng môi trường Mini Pacman với không gian trạng thái, không gian hành động, cơ chế phần thưởng và điều kiện kết thúc phù hợp cho quá trình huấn luyện tác tử.
- Cài đặt Q-Learning, DQN và Double DQN theo các nguyên lý được trình bày trong những công trình nền tảng, đồng thời tổ chức cấu hình, huấn luyện, đánh giá và lưu mô hình cho từng thuật toán [2–5].
- Thực hiện chín cấu hình thí nghiệm, tương ứng với ba thuật toán và ba mức learning rate, trong cùng môi trường và quy trình đánh giá.
- Thu thập và so sánh kết quả bằng các chỉ số reward, completion rate, win rate và steps. Loss chỉ được sử dụng khi phân tích DQN và Double DQN.
- Xây dựng demo các tác tử sau huấn luyện, từ đó kết hợp quan sát định tính với kết quả định lượng để đánh giá ưu điểm, hạn chế của từng phương pháp và giới hạn của thực nghiệm.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là ba thuật toán RL dựa trên giá trị: Q-Learning dạng bảng, DQN và Double DQN. Báo cáo tập trung vào nguyên lý hoạt động, cách cài đặt, quá trình huấn luyện và kết quả thực nghiệm của ba thuật toán trong ứng dụng do nhóm xây dựng.

Thực nghiệm được tiến hành trên bản đồ medium có kích thước 15×15 ô của môi trường Mini Pacman. Tác tử Pacman lựa chọn một trong bốn hướng di chuyển và tương tác với thức ăn, viên sức mạnh, trái cây cùng ba con ma trong mê cung. Q-Learning sử dụng trạng thái rời rạc làm khóa của Q-table, trong khi

DQN và Double DQN nhận vector đặc trưng được tạo từ trạng thái môi trường. Đề tài không nghiên cứu phương pháp học trực tiếp từ ảnh chụp màn hình.

Phạm vi thực nghiệm gồm chín cấu hình được tạo từ ba thuật toán và ba mức learning rate lựa chọn trước. Mỗi cấu hình được huấn luyện với một seed. Tại mỗi thời điểm đánh giá định kỳ, mô hình được chạy trong 20 lượt chơi (episode). Kết quả được xem xét qua reward, completion rate, win rate, steps và loss đối với các thuật toán sử dụng mạng nơ-ron. Đề tài chỉ khảo sát một bản đồ và không thực hiện tìm kiếm siêu tham số trên quy mô lớn. Vì vậy, các kết luận được giới hạn trong môi trường và cấu hình thực nghiệm của đề tài, không được xem là kết luận chung về hiệu quả của các thuật toán trong mọi bài toán.

1.4 Đóng góp của đề tài

Đóng góp chính của đề tài là xây dựng được môi trường Mini Pacman phục vụ quá trình huấn luyện và đánh giá tác tử RL. Môi trường mô phỏng mê cung, thức ăn, viên sức mạnh, trái cây, ba con ma, số mạng và các điều kiện kết thúc lượt chơi. Trên môi trường này, nhóm cài đặt Q-Learning, DQN và Double DQN trong cùng một ứng dụng, đồng thời xây dựng hệ thống cấu hình YAML, lưu checkpoint, lưu mô hình sau huấn luyện và đánh giá định kỳ.

Từ hệ thống đã xây dựng, nhóm thực hiện chín cấu hình huấn luyện tương ứng với ba thuật toán và ba mức learning rate. Kết quả được tổng hợp qua reward, completion rate, win rate, steps và loss khi phù hợp, qua đó hỗ trợ đánh giá các thuật toán từ nhiều khía cạnh. Bên cạnh các kết quả định lượng, nhóm còn xây dựng biểu đồ so sánh và demo các tác tử sau huấn luyện để quan sát trực tiếp hành vi của mô hình trong môi trường trò chơi.

2 CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về học tăng cường

Học tăng cường (reinforcement learning – RL) nghiên cứu cách một tác tử lựa chọn hành động khi tương tác với môi trường để tối đa hóa phần thưởng tích lũy. Khác với học có giám sát, tác tử không nhận sẵn nhãn cho từng quyết định. Thay vào đó, tác tử quan sát kết quả của các hành động đã thực hiện và điều chỉnh cách ra quyết định dựa trên phần thưởng nhận được. Bốn thành phần cơ bản của một hệ thống RL gồm chính sách, tín hiệu phần thưởng, hàm giá trị và mô hình môi trường nếu mô hình đó được biết hoặc được học [1, 8].

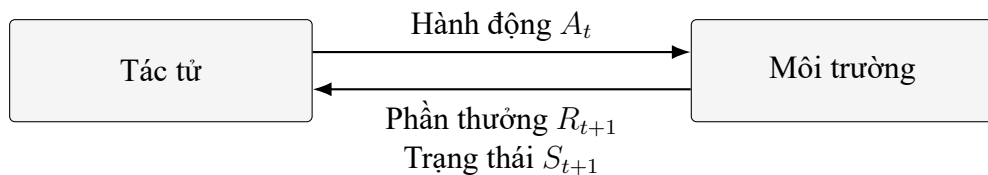
Trong phạm vi đề tài, ba thuật toán Q-Learning, DQN và Double DQN đều thuộc nhóm phương pháp dựa trên giá trị (value-based methods). Các phương pháp này học giá trị của hành động trong từng trạng thái, sau đó ưu tiên hành động có giá trị ước lượng lớn nhất. Điểm khác biệt chính nằm ở cách biểu diễn giá trị hành động và cách xây dựng giá trị mục tiêu trong quá trình cập nhật.

2.1.1 Quá trình quyết định Markov

Quá trình quyết định Markov (Markov Decision Process – MDP) là mô hình toán học cho bài toán ra quyết định tuần tự. Tại thời điểm t , tác tử nhận trạng thái $S_t \in \mathcal{S}$, lựa chọn hành động $A_t \in \mathcal{A}(S_t)$, sau đó môi trường trả về phần thưởng R_{t+1} và trạng thái tiếp theo S_{t+1} . Chuỗi tương tác có dạng

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots \quad (2.1)$$

Mối quan hệ giữa tác tử và môi trường được minh họa trong Hình 2.1. Hành động đi từ tác tử tới môi trường, còn trạng thái và phần thưởng đi theo chiều ngược lại.



Hình 2.1: Tương tác giữa tác tử và môi trường trong MDP [1].

Với MDP hữu hạn, động lực của môi trường có thể được mô tả bằng phân phối xác suất

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}. \quad (2.2)$$

Phân phối này cho biết xác suất chuyển tới trạng thái s' và nhận phần thưởng r khi tác tử thực hiện hành động a tại trạng thái s . Tính chất Markov yêu cầu trạng thái hiện tại phải chứa những thông tin từ quá khứ có ảnh hưởng đến diễn biến tương lai. Khi đã biết S_t và A_t , xác suất của trạng thái cùng phần thưởng tiếp theo không phụ thuộc thêm vào toàn bộ lịch sử trước đó [1].

Một chính sách (policy) được ký hiệu bởi $\pi(a \mid s)$, biểu diễn xác suất lựa chọn hành động a tại trạng thái s . Mục tiêu của tác tử là tìm chính sách tạo ra phần thưởng tích lũy kỳ vọng lớn. Đối với bài toán theo episode như Mini Pacman, mỗi episode kết thúc khi tác tử đạt điều kiện kết thúc của môi trường hoặc chạm giới hạn số bước.

2.1.2 Phần thưởng tích lũy và hàm giá trị

Phần thưởng tức thời chỉ phản ánh kết quả ngay sau một hành động. Để đánh giá ảnh hưởng dài hạn, RL sử dụng phần thưởng tích lũy có chiết khấu, còn gọi là return. Tại thời điểm t , return được định nghĩa là

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad (2.3)$$

trong đó $\gamma \in [0, 1]$ là discount factor. Giá trị γ nhỏ làm tác tử ưu tiên phần thưởng gần, còn giá trị gần 1 làm tăng ảnh hưởng của phần thưởng trong tương lai. Với bài toán theo episode, tổng trên kết thúc tại trạng thái terminal và giá trị sau thời điểm kết thúc được xem bằng 0 [1].

Hàm giá trị trạng thái dưới chính sách π được định nghĩa bởi

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]. \quad (2.4)$$

Đại lượng này cho biết return kỳ vọng khi bắt đầu tại trạng thái s và tiếp tục lựa chọn hành động theo π . Tương tự, hàm giá trị hành động là

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a], \quad (2.5)$$

cho biết return kỳ vọng khi chọn hành động a tại s , sau đó tiếp tục theo chính sách π .

Giá trị hành động tối ưu được ký hiệu là

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a). \quad (2.6)$$

Hàm này thỏa mãn phương trình tối ưu Bellman

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]. \quad (2.7)$$

Phương trình (2.7) là cơ sở của các thuật toán học giá trị. Giá trị của một cặp trạng thái–hành động được liên hệ với phần thưởng tức thời và giá trị tốt nhất có thể đạt được ở trạng thái tiếp theo [1].

2.1.3 Khám phá và khai thác

Trong quá trình học, tác tử phải cân bằng giữa khám phá (exploration) và khai thác (exploitation). Khai thác chọn hành động đang có Q-value lớn nhất, còn khám phá thử các hành động khác để thu thập thêm kinh nghiệm. Nếu chỉ khai thác từ đầu, tác tử có thể duy trì một hành động chưa thực sự tối ưu do các Q-value ban đầu còn thiếu chính xác. Nếu khám phá quá nhiều, tác tử không tận dụng được những gì đã học.

Một chiến lược phổ biến là ϵ -greedy. Giả sử tại trạng thái s chỉ có một hành động tham lam

$$a^* = \arg \max_{a' \in \mathcal{A}(s)} Q(s, a'). \quad (2.8)$$

Xác suất lựa chọn hành động được xác định bởi

$$\pi_{\epsilon}(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{nếu } a = a^*, \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{nếu } a \neq a^*. \end{cases} \quad (2.9)$$

Với xác suất $1 - \epsilon$, tác tử chọn hành động tham lam. Với xác suất ϵ , tác tử chọn ngẫu nhiên trong tập hành động, vì vậy hành động tham lam vẫn có thể được chọn trong nhánh ngẫu nhiên. Trong thực nghiệm, ϵ thường giảm dần để ưu tiên khám phá ở giai đoạn đầu và khai thác nhiều hơn khi các Q-value đã ổn định hơn [1].

2.2 Q-Learning

Q-Learning được sử dụng trong đề tài như một phương pháp học dạng bảng. Thuật toán lưu Q-value của từng cặp trạng thái–hành động trong Q-table và cập nhật giá trị này từ phần thưởng cùng giá trị tốt nhất tại trạng thái tiếp theo. Q-Learning là phương pháp temporal-difference control không cần mô hình môi trường và thuộc nhóm off-policy: hành động tạo dữ liệu có thể được chọn bằng ϵ -greedy, trong khi target sử dụng hành động có Q-value lớn nhất [1, 2].

Sau khi tác tử thực hiện A_t tại S_t , nhận R_{t+1} và chuyển tới S_{t+1} , phần tử tương ứng trong Q-table được cập nhật theo

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right], \quad (2.10)$$

trong đó $\alpha \in (0, 1]$ là learning rate. Các phần tử khác của Q-table không thay đổi tại bước cập nhật này. Nếu S_{t+1} là trạng thái terminal, thành phần giá trị tương lai được đặt bằng 0.

Giá trị mục tiêu của cập nhật có thể viết là

$$Y_t^Q = R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a'), \quad (2.11)$$

và sai số temporal difference là

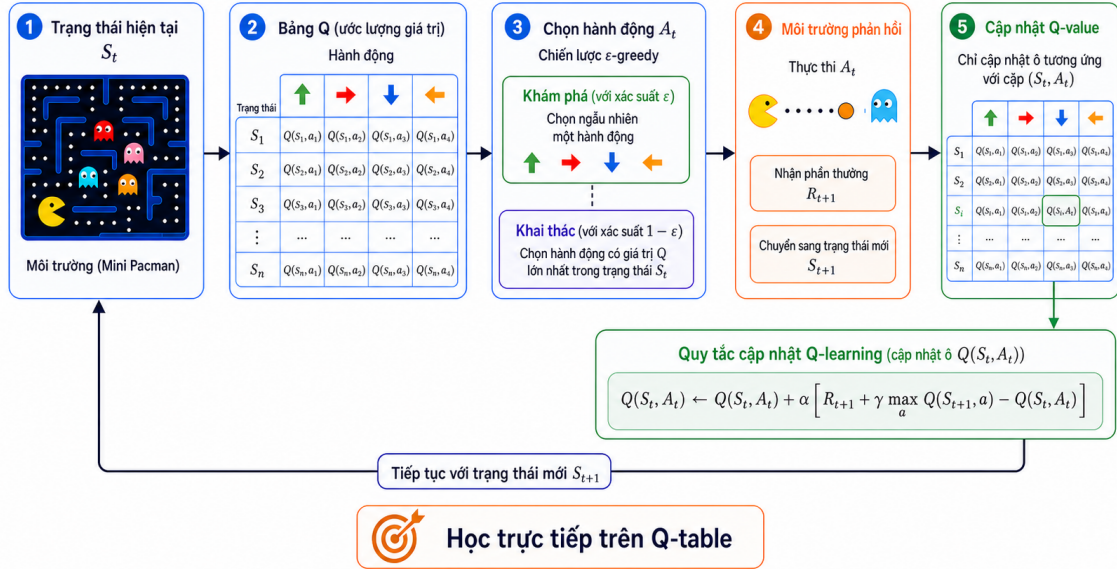
$$\delta_t = Y_t^Q - Q_t(S_t, A_t). \quad (2.12)$$

Như vậy, Q-Learning dịch chuyển Q-value hiện tại một đoạn bằng $\alpha \delta_t$ về phía giá trị mục tiêu.

Quy trình của một bước Q-Learning gồm:

1. Quan sát trạng thái hiện tại S_t .
2. Chọn hành động A_t , thường bằng chiến lược ϵ -greedy.
3. Thực hiện hành động và nhận R_{t+1} cùng S_{t+1} .
4. Tính target theo phương trình (2.11).
5. Cập nhật phần tử $Q(S_t, A_t)$ theo phương trình (2.10).

Quy trình trên được minh họa trong Hình 2.2. Hình nhấn mạnh rằng mỗi bước chỉ cập nhật phần tử Q-value tương ứng với cặp trạng thái–hành động vừa quan sát.



Hình 2.2: Quy trình lựa chọn hành động và cập nhật Q-table của Q-Learning [1, 2].

Về mặt lý thuyết, Q-Learning có thể hội tụ về action-value tối ưu với xác suất 1 khi mọi cặp trạng thái–hành động tiếp tục được lấy mẫu và dãy learning rate thỏa mãn các điều kiện xấp xỉ ngẫu nhiên phù hợp [2]. Kết quả này không bảo đảm rằng mọi quá trình huấn luyện hữu hạn với learning rate cố định đều hội tụ.

Q-table giúp quy trình cập nhật dễ theo dõi nhưng phải lưu riêng Q-value cho từng cặp trạng thái–hành động. Khi số trạng thái tăng, kích thước bảng lớn hơn và các trạng thái khác nhau không có cơ chế chia sẻ thông tin. DQN xử lý hạn chế này bằng cách sử dụng mạng nơ-ron để xấp xỉ Q-value.

2.3 Deep Q-Network

Deep Q-Network (DQN) mở rộng Q-Learning bằng cách sử dụng mạng nơ-ron để xấp xỉ hàm giá trị hành động thay cho Q-table. Quá trình huấn luyện kết hợp experience replay nhằm tái sử dụng các chuyển tiếp và target network nhằm giữ giá trị mục tiêu ổn định hơn [3, 4].

2.3.1 Mạng Q

Thay vì lưu một Q-table, DQN sử dụng mạng nơ-ron có tham số θ để xấp xỉ

$$Q(s, a; \theta) \approx q_*(s, a). \quad (2.13)$$

Với một trạng thái đầu vào, mạng Q xuất ra vector giá trị

$$\mathbf{Q}(s; \theta) = [Q(s, a_1; \theta), Q(s, a_2; \theta), \dots, Q(s, a_m; \theta)], \quad (2.14)$$

trong đó m là số hành động. Khi khai thác, tác tử chọn hành động có phần tử đầu ra lớn nhất. Trong quá trình khám phá, hành động vẫn được chọn theo chính sách ϵ -greedy.

Q-network có thể nhận nhiều dạng biểu diễn trạng thái khác nhau. Trong đề tài, đầu vào của mạng là vector đặc trưng được xây dựng từ trạng thái Mini Pacman, không phải chuỗi khung hình như cấu hình Atari [3, 4].

Do các trạng thái dùng chung tham số θ , một lần cập nhật có thể làm thay đổi Q-value ước lượng của nhiều trạng thái. Điều này tạo khả năng khái quát hóa mà Q-table không có sẵn, nhưng cũng khiến target thay đổi trong quá trình học và có thể làm huấn luyện mất ổn định. DQN sử dụng experience replay và target network để giảm các vấn đề này.

2.3.2 Experience replay

Tại mỗi bước, tác tử thu được một chuyển tiếp

$$e_t = (S_t, A_t, R_{t+1}, S_{t+1}). \quad (2.15)$$

Các chuyển tiếp được lưu trong replay buffer $\mathcal{D}$. Khi cập nhật mạng, DQN không chỉ sử dụng chuyển tiếp mới nhất mà lấy ngẫu nhiên một mini-batch từ buffer. Với bài toán theo episode, cài đặt thực tế thường lưu thêm cờ kết thúc để xác định target có chứa giá trị tương lai hay không.

Experience replay có ba vai trò chính. Thứ nhất, một chuyển tiếp có thể được tái sử dụng trong nhiều lần cập nhật, giúp sử dụng dữ liệu hiệu quả hơn. Thứ hai, lấy mẫu ngẫu nhiên làm giảm tương quan mạnh giữa các chuyển tiếp liên tiếp. Thứ ba, dữ liệu trong buffer được tạo bởi nhiều phiên bản chính sách ở các thời điểm khác nhau, giúp làm mượt sự thay đổi của phân phối huấn luyện [3, 4].

Replay buffer có dung lượng hữu hạn nên các chuyển tiếp cũ sẽ bị thay thế. Trong phạm vi đề tài, các chuyển tiếp được lấy mẫu đều từ buffer. Prioritized experience replay không được xem xét.

2.3.3 Target network và loss

DQN duy trì hai mạng có cùng kiến trúc:

- Online network với tham số θ được cập nhật bằng gradient descent.
- Target network với tham số θ^- được dùng để tính giá trị mục tiêu.

Sau một số bước cập nhật cố định, tham số của online network được sao chép sang target network. Giữa hai lần đồng bộ, θ^- được giữ nguyên. Nhờ đó, target không thay đổi ngay sau từng lần cập nhật online network, làm giảm vòng phản hồi có thể gây dao động hoặc phân kỳ [4].

Với một chuyển tiếp, target của DQN được xác định bởi

$$Y_t^{\text{DQN}} = \begin{cases} R_{t+1}, & \text{nếu } S_{t+1} \text{ là terminal,} \\ R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta^-), & \text{trường hợp còn lại.} \end{cases} \quad (2.16)$$

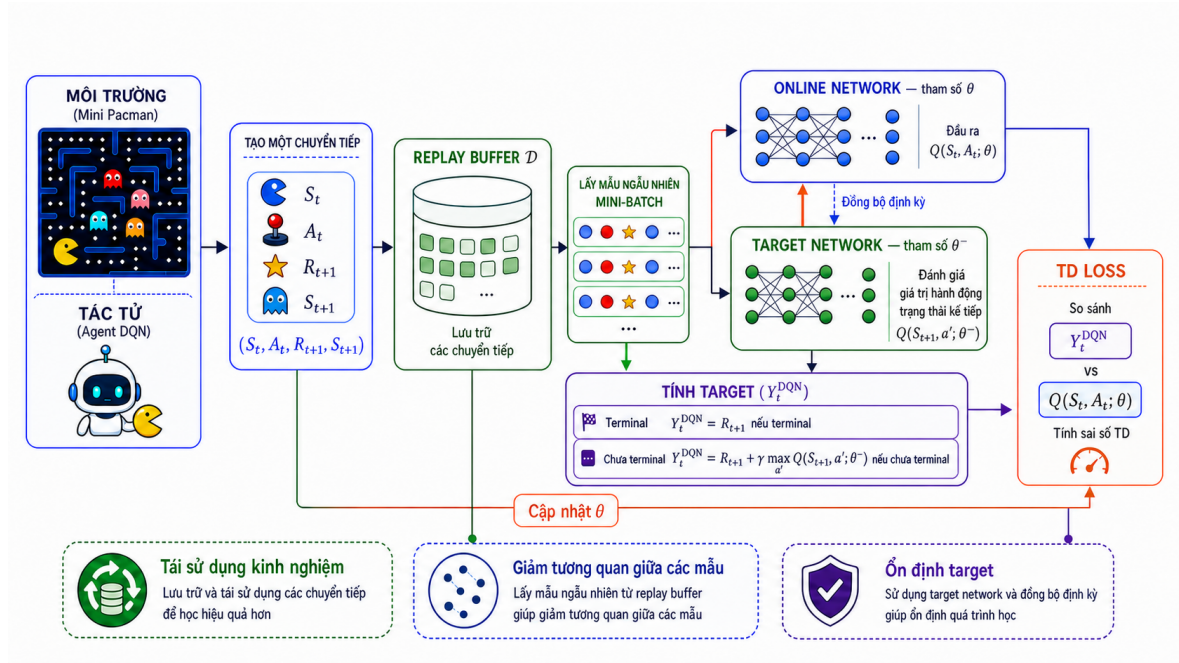
Online network tạo ra giá trị hiện tại $Q(S_t, A_t; \theta)$, còn target network tạo thành phần giá trị ở trạng thái tiếp theo.

Trong cài đặt của đề tài, online network được tối ưu bằng Smooth L1 Loss với ngưỡng chuyển tiếp bằng 1. Gọi δ_i là TD error của mẫu thứ i , loss được xác định bởi

$$\begin{aligned} \delta_i &= Y_i^{\text{DQN}} - Q(s, a; \theta_i), \\ L_i(\theta_i) &= \mathbb{E}_{(s,a,r,s') \sim U(\mathcal{D})} [\ell(\delta_i)], \\ \ell(\delta) &= \begin{cases} \frac{1}{2}\delta^2, & \text{nếu } |\delta| < 1, \\ |\delta| - \frac{1}{2}, & \text{trường hợp còn lại.} \end{cases} \end{aligned} \quad (2.17)$$

So với bình phương TD error, Smooth L1 Loss vẫn phạt sai lệch nhỏ theo dạng bậc hai nhưng chuyển sang dạng tuyến tính khi sai lệch lớn. Cách tính này đúng với hàm SmoothL1Loss được sử dụng trong mã nguồn của đề tài [7]. Learning rate của optimizer điều khiển độ lớn bước cập nhật tham số θ của online network khi tối ưu loss.

Luồng dữ liệu huấn luyện DQN được tổng hợp trong Hình 2.3. Online network được cập nhật qua loss, còn target network chỉ nhận tham số qua bước đồng bộ định kỳ.



Hình 2.3: Luồng huấn luyện DQN với experience replay và target network [3,4].

2.4 Double DQN

Target của DQN trong phương trình (2.16) sử dụng cùng các giá trị do target network sinh ra để vừa chọn hành động có giá trị lớn nhất, vừa đánh giá hành động đó. Khi các Q-value chỉ là ước lượng và chứa sai số, phép lấy maximum có xu hướng ưu tiên những hành động đang bị đánh giá cao hơn giá trị thực. Hiện tượng này được gọi là overestimation [5].

Đối với chuyển tiếp không kết thúc episode, target DQN có thể viết lại thành

$$Y_t^{\text{DQN}} = R_{t+1} + \gamma Q \left(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a'; \theta_t^-); \theta_t^- \right). \quad (2.18)$$

Cả lựa chọn và đánh giá hành động đều dựa trên tham số θ_t^- .

Double DQN tách hai vai trò này. Online network chọn hành động tham lam

$$a_{t+1}^* = \arg \max_{a'} Q(S_{t+1}, a'; \theta_t), \quad (2.19)$$

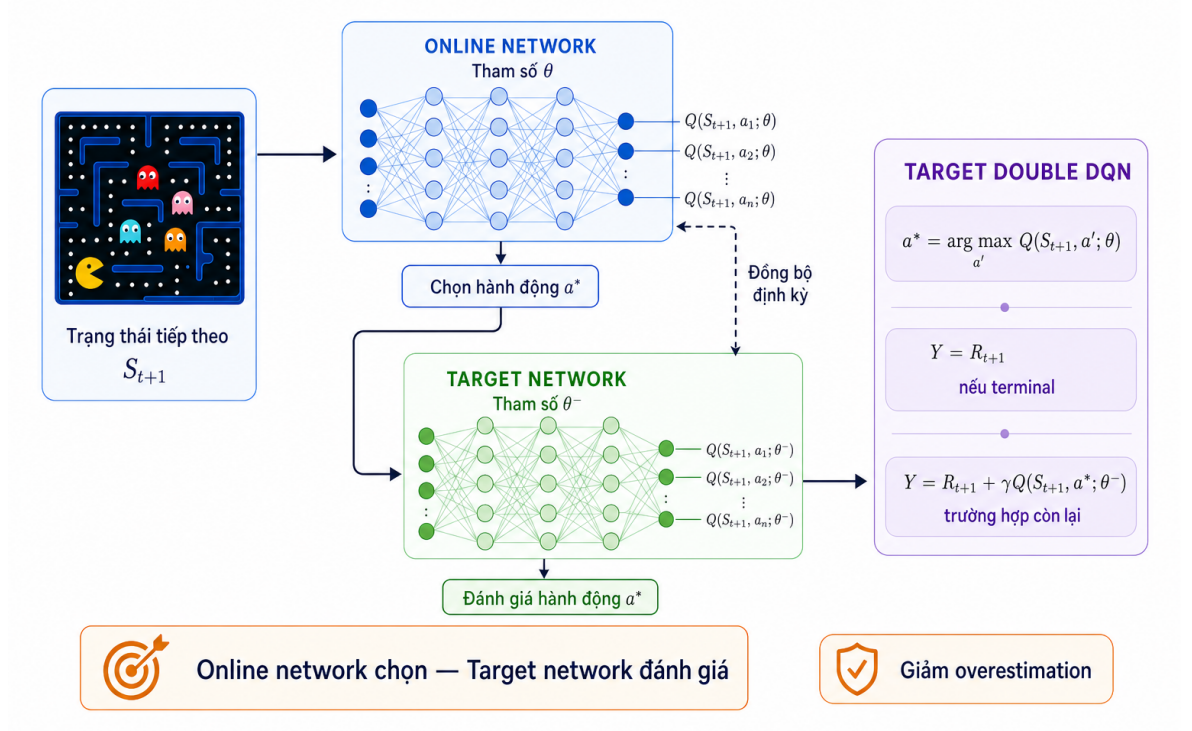
sau đó target network đánh giá hành động đã chọn. Target Double DQN là

$$Y_t^{\text{DDQN}} = \begin{cases} R_{t+1}, & \text{nếu } S_{t+1} \text{ là terminal,} \\ R_{t+1} + \gamma Q(S_{t+1}, a_{t+1}^*; \theta_t^-), & \text{trường hợp còn lại.} \end{cases} \quad (2.20)$$

Loss của Double DQN giữ nguyên dạng của phương trình (2.17), nhưng Y_i^{DQN} được thay bằng Y_i^{DDQN} khi tính TD error [5].

Sự khác biệt nằm ở thành phần target. DQN lấy maximum trực tiếp từ target network, còn Double DQN dùng online network để chọn hành động và target network để đánh giá. Target network vẫn được đồng bộ định kỳ, còn optimizer và learning rate được giữ nguyên như DQN. Vì vậy, Double DQN không cần thêm mạng thứ ba và giữ nguyên phần còn lại của quy trình huấn luyện [5].

Nguyên lý tách vai trò của Double DQN được minh họa trong Hình 2.4. Online network chọn hành động tham lam tại trạng thái tiếp theo, còn target network đánh giá chính hành động đã chọn.



Hình 2.4: Cơ chế lựa chọn và đánh giá hành động khi tính target Double DQN [5].

Double DQN làm giảm nhưng không loại bỏ hoàn toàn overestimation. Hiệu

quả của cơ chế này vẫn phụ thuộc vào dữ liệu, biểu diễn trạng thái, kiến trúc mạng và cấu hình huấn luyện [5].

2.5 So sánh Q-Learning, DQN và Double DQN

Bảng 2.1 tổng hợp những khác biệt quan trọng về nguyên lý của ba thuật toán.

Tiêu chí	Q-Learning	DQN	Double DQN
Biểu diễn Q-value	Q-table với một phần tử cho mỗi cặp trạng thái-hành động.	Mạng Q có tham số θ .	Online network và target network như DQN.
Dữ liệu cập nhật	Cập nhật trực tiếp từ từng chuyển tiếp.	Mini-batch lấy ngẫu nhiên từ replay buffer.	Mini-batch lấy ngẫu nhiên từ replay buffer.
Target ở trạng thái tiếp theo	Maximum của hàng Q-table tại trạng thái tiếp theo.	Target network vừa chọn vừa đánh giá hành động có giá trị lớn nhất.	Online network chọn hành động, target network đánh giá hành động đó.
Khái quát hóa	Không có cơ chế chia sẻ giá trị giữa các trạng thái khác nhau.	Có khả năng chia sẻ thông tin thông qua tham số mạng.	Tương tự DQN.
Loss của mạng nơ-ron	Không sử dụng.	Tối ưu TD error giữa Q-value hiện tại và target DQN.	Tối ưu TD error với target Double DQN.
Hạn chế chính	Q-table tăng nhanh khi không gian trạng thái lớn.	Huấn luyện có thể mất ổn định và xuất hiện overestimation.	Giảm overestimation nhưng vẫn phụ thuộc vào hàm xấp xỉ và cấu hình huấn luyện.

Bảng 2.1: So sánh Q-Learning, DQN và Double DQN về mặt nguyên lý.

Q-Learning lưu trực tiếp Q-value trong Q-table. DQN sử dụng mạng nơ-ron để xấp xỉ Q-value, còn Double DQN giữ nguyên cơ chế huấn luyện của DQN nhưng tách quá trình lựa chọn và đánh giá hành động khi tính target.

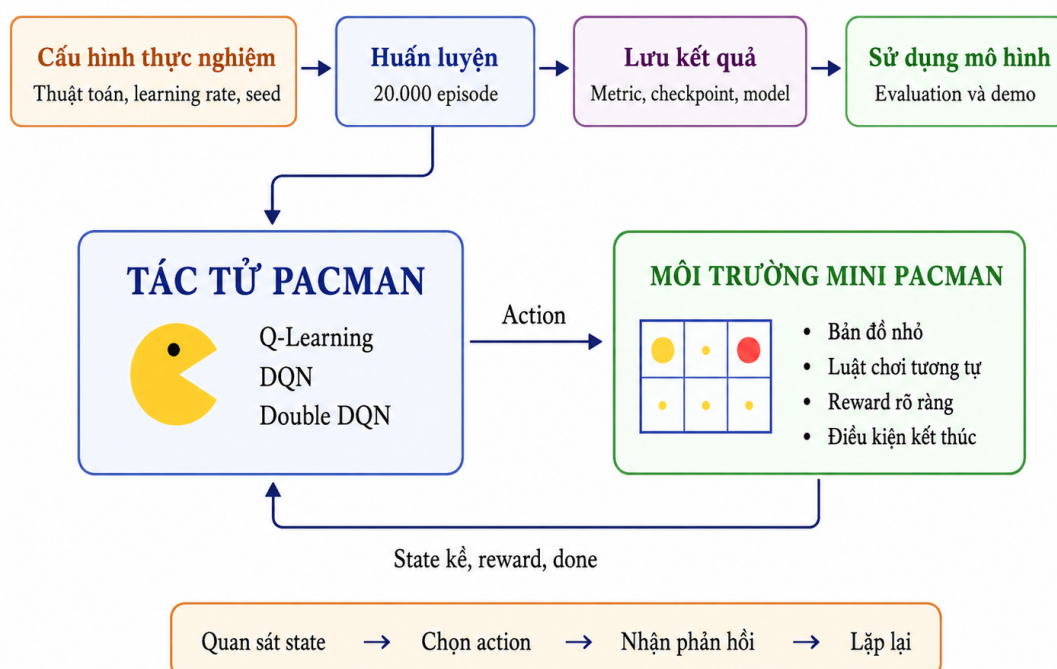
Từ các cơ sở lý thuyết trên, chương tiếp theo trình bày cách nhóm cụ thể hóa bài toán Mini Pacman thành môi trường học tăng cường, bao gồm biểu diễn trạng thái, không gian hành động, cơ chế phần thưởng và điều kiện kết thúc. Trên nền tảng đó, Q-Learning, DQN và Double DQN được cài đặt trong cùng một hệ thống và đánh giá theo một quy trình thực nghiệm thống nhất.

3 XÂY DỰNG HỆ THỐNG VÀ THIẾT KẾ THỰC NGHIỆM

3.1 Tổng quan ứng dụng học tăng cường trong trò chơi Pacman

Đề tài xây dựng môi trường Mini Pacman để huấn luyện tác tử bằng ba phương pháp Q-Learning, DQN và Double DQN. Cả ba phương pháp cùng sử dụng một bản đồ, một bộ luật và một hệ tiêu chí đánh giá. Điểm khác biệt nằm ở cách biểu diễn và cập nhật hàm giá trị. Q-Learning lưu các Q-value trong Q-table, còn DQN và Double DQN xấp xỉ chúng bằng Q-network. Riêng DQN và Double DQN khác nhau ở cách xác định target cho bước cập nhật [7].

Hình 3.1 trình bày kiến trúc tổng thể của hệ thống. Trong mỗi bước tương tác, tác tử nhận state từ môi trường, chọn action rồi nhận reward, state kế tiếp và cờ kết thúc. Quá trình huấn luyện được điều khiển bởi cấu hình thực nghiệm và định kỳ lưu checkpoint cùng kết quả đánh giá. Khi huấn luyện kết thúc, mô hình cuối được dùng cho evaluation và demo mà không cập nhật thêm tham số.



Hình 3.1: Kiến trúc tổng thể của ứng dụng học tăng cường trong trò chơi Pacman.

3.2 Môi trường Mini Pacman

Môi trường Mini Pacman tạo ra cùng một bài toán điều khiển cho cả ba phương pháp. Mỗi episode bắt đầu từ trạng thái khởi tạo của bản đồ. Sau khi nhận action, môi trường cập nhật trò chơi rồi trả về state kế tiếp, reward và cờ kết thúc. Tất cả thí nghiệm đều sử dụng bản đồ medium, ba mạng sống, ba ma và giới hạn 700 bước cho mỗi episode [7].

3.2.1 Bản đồ và các thành phần trong môi trường

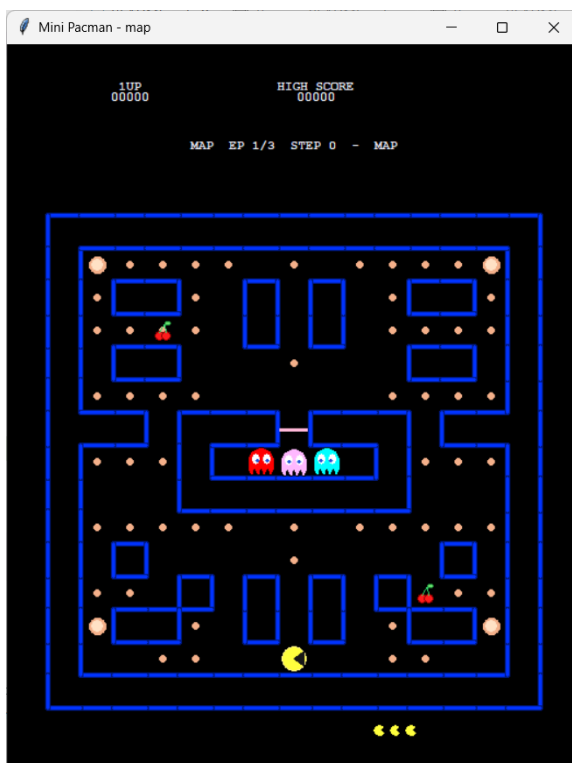
Bản đồ medium có kích thước 15×15 , gồm tường, hành lang, ghost door và khu vực xuất phát của ba ma ở trung tâm. Pacman xuất hiện ở phần dưới bản đồ. Hệ thống hành lang có nhiều ngã rẽ, buộc tác tử phải cân bằng giữa việc thu thập food, tránh ma và lựa chọn đường đi phù hợp.

Bản đồ chứa 62 food, trong đó bốn food gần các góc là power pellet. Ngoài ra, hai bonus fruit được đặt tại các vị trí cố định. Power pellet vẫn được tính vào tổng số food cần thu thập. Bonus fruit chỉ mang lại reward bổ sung và không ảnh hưởng đến điều kiện chiến thắng. Bảng 3.1 tổng hợp các thành phần chính của môi trường.

Thành phần	Số lượng	Vai trò trong môi trường
Bản đồ	15×15 ô	Gồm tường, hành lang, ghost door và khu vực xuất phát của ma.
Pacman	1	Tác tử được huấn luyện, bắt đầu với ba mạng sống.
Ma	3	Blinky, Pinky và Inky, xuất phát tại khu vực trung tâm.
Food	62	Phân bố trên hành lang và là điều kiện để hoàn thành bản đồ.
Power pellet	4	Nằm trong 62 food và kích hoạt trạng thái frightened.
Bonus fruit	2	Nằm ở vị trí cố định và cung cấp thêm reward.

Bảng 3.1: Các thành phần của bản đồ medium [7].

Hình 3.2 minh họa trạng thái ban đầu của bản đồ trong một lượt demo. Bố cục bản đồ được giữ nguyên ở cả ba giai đoạn training, evaluation và demo.



Hình 3.2: Bản đồ medium của môi trường Mini Pacman trong một lượt chạy demo.

3.2.2 Luật hoạt động và điều kiện kết thúc

Trong mỗi environment step, Pacman di chuyển trước theo action đã chọn. Nếu hướng đi bị tường hoặc ghost door chặn, Pacman đứng tại chỗ và nhận penalty va tường. Sau đó, môi trường xử lý việc thu thập food, power pellet, bonus fruit và các va chạm tại vị trí mới. Ba ma tiếp tục thực hiện lượt di chuyển. Va chạm được kiểm tra thêm một lần trước khi môi trường trả về state mới, reward và cờ kết thúc.

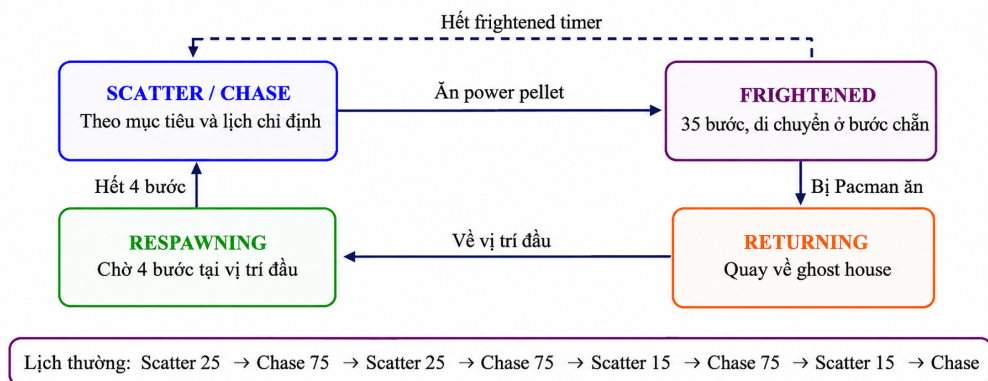
Khi không frightened, ba ma luân phiên giữa scatter và chase. Mỗi ma sử dụng một mục tiêu riêng trong từng chế độ như trình bày trong Bảng 3.2.

Ma	Mục tiêu trong chế độ chase	Mục tiêu trong chế độ scatter
Blinky (đỏ)	Đuổi trực tiếp tới vị trí hiện tại của Pacman.	Góc trên bên phải của bản đồ, tại vị trí (1, 13).
Pinky (hồng)	Nhắm tới điểm nằm trước Pacman tối đa bốn ô theo hướng di chuyển hiện tại để chặn đầu.	Góc trên bên trái của bản đồ, tại vị trí (1, 1).
Inky (xanh lam)	Lấy điểm nằm trước Pacman tối đa hai ô, sau đó kéo dài vector từ Blinky tới điểm này thêm một lần để tạo mục tiêu.	Góc dưới bên phải của bản đồ, tại vị trí (13, 13).

Bảng 3.2: Mục tiêu di chuyển của ba ma trong chế độ chase và scatter [7].

Tại mỗi bước, ma loại hướng quay ngược nếu vẫn còn hướng hợp lệ khác, sau đó ưu tiên hướng tạo ra maze distance nhỏ nhất tới mục tiêu. Nếu nhiều hướng có cùng khoảng cách tốt nhất, hệ thống lựa chọn bằng bộ sinh số ngẫu nhiên đã được cố định seed. Cách xác định mục tiêu làm Blinky bám trực tiếp Pacman, Pinky tìm cách chặn đầu, còn Inky phối hợp vị trí của Blinky với hướng di chuyển của Pacman.

Power pellet đặt frightened timer bằng 35 bước. Trong thời gian này, ma chỉ di chuyển ở các environment step chẵn và ưu tiên hướng làm tăng maze distance tới Pacman. Nếu bị Pacman ăn, ma chuyển sang returning, quay về vị trí xuất phát rồi chờ bốn bước trước khi hoạt động trở lại. Các trạng thái chính và điều kiện chuyển đổi được minh họa trong Hình 3.3.



Hình 3.3: Các trạng thái hoạt động và quá trình chuyển đổi của ma trong môi trường Mini Pacman.

Episode kết thúc khi Pacman ăn hết 62 food, mất mạng cuối cùng hoặc đạt

giới hạn 700 bước. Nếu Pacman bị bắt nhưng vẫn còn mạng, episode tiếp tục: Pacman và ba ma trở về vị trí xuất phát, còn food và bonus fruit đã ăn không được khôi phục. Do đó, tiến độ thu thập vật phẩm được giữ lại qua các lần mất mạng trong cùng một episode.

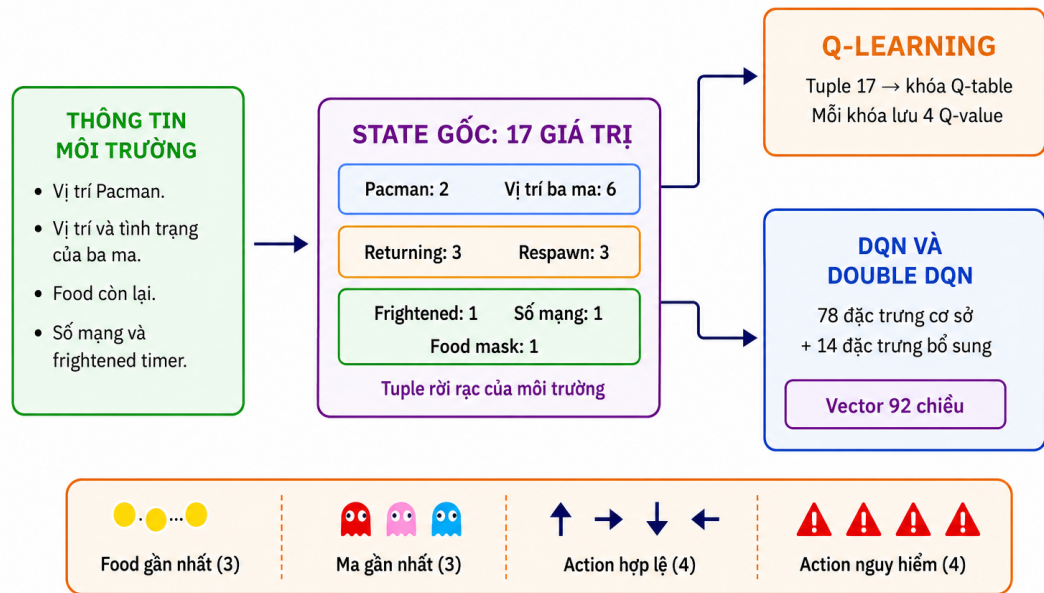
3.3 Mô hình hóa bài toán học tăng cường

Để lựa chọn action, tác tử cần nắm được vị trí của Pacman, vị trí và trạng thái của các ma, lượng food còn lại, số mạng và thời gian hiệu lực của power pellet. Môi trường mã hóa những thông tin này theo hai cách. Q-Learning sử dụng state dạng tuple, còn DQN và Double DQN nhận state dưới dạng vector số thực.

3.3.1 Không gian và biểu diễn trạng thái

State gốc gồm 17 giá trị, bao gồm hai tọa độ của Pacman, sáu tọa độ của ba ma, ba cờ returning, ba respawn timer, frightened timer, số mạng còn lại và một food mask. Q-Learning dùng trực tiếp tuple này làm khóa của Q-table. Mỗi khóa gắn với bốn Q-value tương ứng với bốn action. Các giá trị được khởi tạo bằng 0 khi state xuất hiện lần đầu.

Đối với DQN và Double DQN, state được mở rộng thành vector 92 chiều. Food mask được tách thành 62 bit rồi ghép thêm thông tin về food gần nhất, ma đang hoạt động gần nhất, action hợp lệ và action nguy hiểm. Hai cách biểu diễn state được minh họa trong Hình 3.4. Cấu trúc chi tiết của vector 92 chiều được trình bày trong Bảng 3.3.



Hình 3.4: Hai cách biểu diễn state được sử dụng cho Q-Learning và các phương pháp deep Q-learning.

Nhóm đặc trưng	Số chiều	Cách biểu diễn
Vị trí Pacman	2	Tọa độ hàng và cột, mỗi giá trị chia cho 14.
Vị trí ba ma	6	Hai tọa độ cho mỗi ma, mỗi giá trị chia cho 14.
Returning flags	3	Một cờ 0/1 cho mỗi ma.
Respawn timers	3	Một bộ đếm cho mỗi ma, chia cho 4.
Số mạng còn lại	1	Số mạng chia cho 3.
Frightened timer	1	Bộ đếm frightened chia cho 35.
Food mask	62	Một cờ 0/1 cho mỗi vị trí food cố định.
Food gần nhất	3	Độ lệch hàng, độ lệch cột chia cho 14 và khoảng cách Manhattan chia cho 28.
Ma đang hoạt động gần nhất	3	Độ lệch hàng, độ lệch cột chia cho 14 và khoảng cách Manhattan chia cho 28.
Action hợp lệ	4	Bốn cờ 0/1 theo thứ tự lên, phải, xuống và trái.
Action nguy hiểm	4	Bốn cờ 0/1, cho biết vị trí sau action có cách ma không quá hai ô Manhattan hay không.
Tổng cộng	92	78 đặc trưng cơ sở và 14 đặc trưng bổ sung.

Bảng 3.3: Cấu trúc vector state của DQN và Double DQN [7].

Food và ma gần nhất được lựa chọn theo maze distance, nhưng thành phần

khoảng cách tương đối lưu trong vector là Manhattan distance. Khi xác định ma gần nhất, các ma đang returning hoặc respawning được loại bỏ. Nếu frightened timer lớn hơn 0, bốn cờ action nguy hiểm đều bằng 0. Power pellet không cần nhóm riêng vì bốn vị trí này cố định trong 62 food bit. Ảnh hưởng sau khi ăn được thể hiện qua frightened timer. Tường và ghost door cũng không được mã hóa thành toàn bộ bản đồ vì bố cục không thay đổi, trong khi bốn cờ action hợp lệ cung cấp thông tin di chuyển tại vị trí hiện tại [7].

Biểu diễn trên không chứa tình trạng của bonus fruit, số bước hiện tại, chế độ scatter/chase, action trước đó của Pacman và action trước đó của các ma. Các biến này vẫn tham gia vào luật chuyển trạng thái hoặc điều kiện kết thúc. Vì vậy, state được lựa chọn đáp ứng phạm vi thực nghiệm của đề tài nhưng không mô tả toàn bộ biến nội bộ của môi trường. Đầu vào của cả ba phương pháp được tạo từ dữ liệu trạng thái, không sử dụng ảnh chụp màn hình.

3.3.2 Không gian hành động

Không gian action gồm bốn hướng theo thứ tự lên, phải, xuống và trái. Tác tử không có action đứng yên. Cả Q-Learning, DQN và Double DQN đều có thể chọn một trong bốn action, kể cả khi hướng đó bị tường hoặc ghost door chặn. Trong trường hợp này, Pacman giữ nguyên vị trí và nhận penalty va tường. Bốn cờ action hợp lệ trong vector state chỉ cung cấp thông tin cho Q-network, không giới hạn trực tiếp action được chọn.

Trong training, action được chọn theo ϵ -greedy. Ở exploration, tác tử chọn ngẫu nhiên một trong bốn action. Ở exploitation, action có Q-value lớn nhất được sử dụng. Trong evaluation, ϵ được đặt bằng 0 để loại bỏ exploration.

3.3.3 Thiết kế phần thưởng

Reward vừa phản ánh kết quả của các sự kiện trong trò chơi, vừa định hướng Pacman thu thập food và duy trì khoảng cách an toàn với ma. Các thành phần có thể được cộng trong cùng một environment step. Chẳng hạn, khi Pacman ăn power pellet, reward của food và reward bổ sung của power pellet cùng được ghi nhận. Các giá trị chính xác được trình bày trong Bảng 3.4.

Thành phần reward	Giá trị cộng thêm
Mỗi environment step	-0,10
Va vào tường hoặc ghost door	-1,00
Ăn một food	+10,00
Food là power pellet	+50,00
Ăn một bonus fruit	+100,00
Ăn một ma đang frightened	+200,00
Ăn hết food và thắng episode	+100,00
Bị ma bắt và mất một mạng	-50,00
Không thắng và không bị ma bắt trong bước hiện tại	+0,02
Tiến gần food gần nhất khi chưa ăn vật phẩm	+0,03
Đi xa food gần nhất khi chưa ăn vật phẩm	-0,03
Tăng khoảng cách tới ma gần nhất	+0,05
Tiến gần ma trong vùng nguy hiểm không quá năm ô	-0,10
Chạm giới hạn 700 bước	-10,00

Bảng 3.4: Các thành phần reward của môi trường Mini Pacman [7].

Reward tiến gần hoặc đi xa food chỉ được áp dụng khi Pacman không ăn food hay bonus fruit ở bước hiện tại và trên bản đồ vẫn còn food. Reward tồn tại được cộng sau khi kiểm tra điều kiện thắng và bị bắt, nhưng trước khi kiểm tra giới hạn 700 bước. Vì vậy, bước chạm giới hạn vẫn có thể nhận +0,02 trước khi nhận penalty timeout. Các thành phần reward dựa trên khoảng cách tới food và ma sử dụng maze distance. Riêng cờ action nguy hiểm trong vector state sử dụng Manhattan distance sau khi thử action.

3.4 Cài đặt các thuật toán

Cả ba phương pháp đều nhận cùng loại phản hồi từ môi trường nhưng khác nhau ở cơ chế học. Q-Learning cập nhật trực tiếp Q-table sau mỗi transition. DQN và Double DQN đưa transition vào replay buffer rồi lấy mẫu theo mini-batch để cập nhật Q-network [7].

3.4.1 Q-Learning

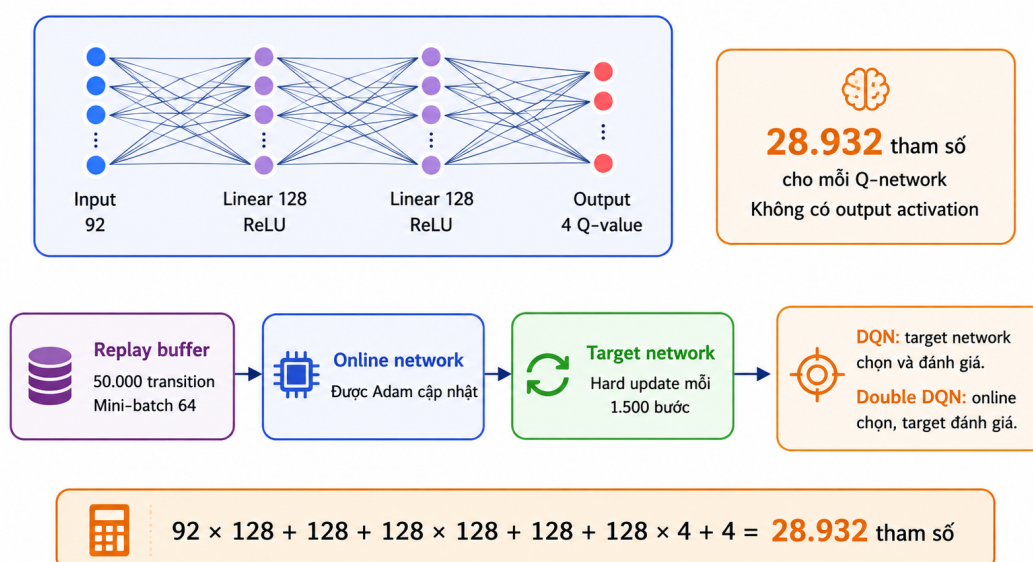
Nguyên lý lựa chọn action và cập nhật Q-table đã được trình bày tại Mục 2.2, đồng thời được minh họa trong Hình 2.2. Khi áp dụng vào Mini Pacman, tuple state gồm 17 giá trị được dùng làm khóa của Q-table. Mỗi khóa lưu bốn Q-value

tương ứng với bốn action. Sau mỗi transition, Q-value của cặp state–action vừa thực hiện được cập nhật theo phương trình (2.10). Nếu transition kết thúc episode, thành phần bootstrap bằng 0. Ngược lại, target sử dụng Q-value lớn nhất của state kế tiếp.

Q-Learning lựa chọn action theo chiến lược ϵ -greedy và giảm epsilon sau mỗi episode. Tại từng mốc checkpoint, hệ thống lưu Q-table, epsilon, trạng thái của bộ sinh số ngẫu nhiên và tiến độ huấn luyện. Nhờ đó, thí nghiệm có thể tiếp tục từ mốc gần nhất nếu bị gián đoạn. Mô hình cuối chỉ chứa Q-table đã học để phục vụ evaluation và demo.

3.4.2 DQN

DQN sử dụng mạng multilayer perceptron với đầu vào là vector state 92 chiều và đầu ra là bốn Q-value. Mạng có hai hidden layer, mỗi layer gồm 128 neuron và sử dụng hàm kích hoạt ReLU. Output layer không dùng hàm kích hoạt. Online network và target network có cùng kiến trúc, mỗi mạng gồm 28.932 tham số. Cấu trúc mạng và mối quan hệ giữa hai nhánh được minh họa trong Hình 3.5.



Hình 3.5: Kiến trúc Q-network và vai trò của online network, target network trong hai phương pháp.

Mỗi transition gồm state vector, action, reward, next-state vector và cờ kết thúc. Các transition được lưu trong replay buffer hoạt động theo cơ chế FIFO

với sức chứa 50.000 mẫu. Việc học bắt đầu sau khi tác tử tích lũy đủ 10.000 environment step. Ở mỗi lần cập nhật, hệ thống lấy ngẫu nhiên 64 transition không lặp trong cùng mini-batch để tối ưu online network. Sau mỗi 1.500 environment step, toàn bộ trọng số của online network được sao chép sang target network.

Online network được tối ưu bằng Adam với learning rate tương ứng của từng cấu hình. Hàm SmoothL1Loss được áp dụng cho TD error, còn gradient norm được giới hạn ở 10,0. DQN target được tính theo phương trình (2.16). Epsilon chỉ giảm sau khi online network thực hiện một lần cập nhật thành công. Vì vậy, epsilon được giữ nguyên trong 10.000 bước thu thập dữ liệu ban đầu.

Mô hình cuối chỉ lưu state dictionary của online network. Khi mô hình được tải để evaluation hoặc demo, target network được đồng bộ lại từ online network. Trong khi đó, checkpoint phục vụ việc tiếp tục training nên lưu thêm target network, optimizer, replay buffer, epsilon, trạng thái bộ sinh số ngẫu nhiên và tiến độ huấn luyện.

3.4.3 Double DQN

Double DQN giữ nguyên vector đầu vào, kiến trúc mạng, optimizer, loss, replay buffer, chiến lược exploration và lịch cập nhật target network của DQN. Điểm khác biệt duy nhất là cách tính target cho next state. Online network chọn action theo phương trình (2.19). Sau đó, target network đánh giá action đã chọn theo phương trình (2.20). TD error vẫn được tối ưu bằng SmoothL1Loss, nhưng DQN target được thay bằng Double DQN target. Các siêu tham số còn lại được giữ giống DQN tại cùng learning rate.

3.5 Thiết kế thực nghiệm

Thực nghiệm gồm chín cấu hình, được tạo từ ba thuật toán và ba giá trị learning rate. Trong phạm vi từng thuật toán, chỉ learning rate được thay đổi. Bản đồ, số episode, seed, lịch evaluation và những thiết lập còn lại được giữ cố định. Cách bố trí này giúp đánh giá ảnh hưởng của learning rate mà không làm thay đổi đồng thời các yếu tố khác.

3.5.1 Cấu hình và siêu tham số

Ba giá trị learning rate được khảo sát là 0,0001, 0,0005 và 0,001. Trong Q-Learning, tham số này điều khiển trực tiếp mức cập nhật của Q-table. Với DQN và Double DQN, learning rate thuộc optimizer Adam. Do hai cơ chế học khác nhau, kết quả chỉ được so sánh giữa các learning rate trong cùng một thuật toán. Một giá trị giống nhau không có nghĩa là Q-table và Q-network thay đổi với mức độ tương đương.

- **Thiết bị huấn luyện:** laptop cá nhân MSI Cyborg 15, trang bị GPU NVIDIA GeForce RTX 4050 Laptop GPU với 6 GB VRAM.

Các tham số chung của chín thí nghiệm được trình bày trong Bảng 3.5.

Tham số	Giá trị
Learning rate	0,0001, 0,0005 và 0,001
Số episode training	20.000
Discount factor	0,99
Layout	medium, 15×15 ô
Số bước tối đa mỗi episode	700
Số mạng sống	3
Số ma	3
Training seed	42
Checkpoint interval	1.000 episode
Evaluation interval	1.000 episode
Số episode mỗi lần evaluation	20
Evaluation seed	10042

Bảng 3.5: Các tham số chung của chín cấu hình thực nghiệm [7].

Các thiết lập riêng của từng phương pháp được tổng hợp trong Bảng 3.6. DQN và Double DQN dùng cùng cấu hình. Sự khác biệt giữa chúng nằm ở cách tính target.

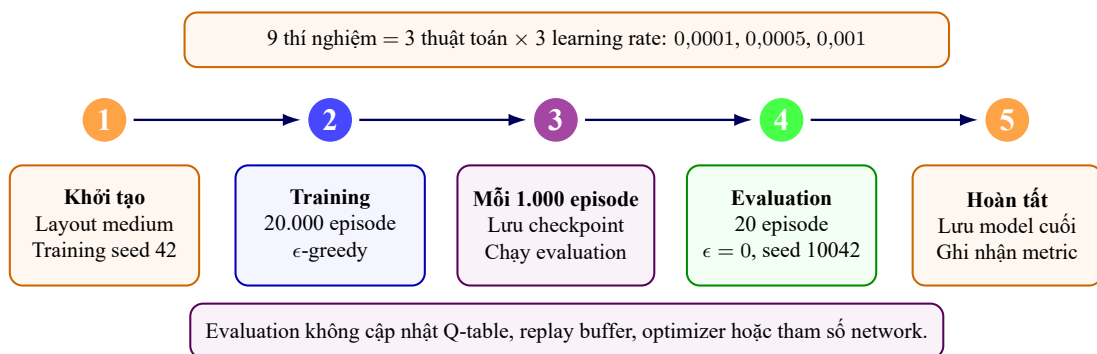
Tham số	Q-Learning	DQN	Double DQN
Epsilon ban đầu	1,0	1,0	1,0
Epsilon nhỏ nhất	0,05	0,02	0,02
Epsilon decay	0,99985/episode	0,999997/update	0,999997/update
Kích thước hidden layer	Không sử dụng	128, 128	128, 128
Batch size	Không sử dụng	64	64
Replay capacity	Không sử dụng	50.000	50.000
Learning starts	Không sử dụng	10.000 bước	10.000 bước
Target update interval	Không sử dụng	1.500 bước	1.500 bước
Optimizer	Không sử dụng	Adam	Adam
Loss	Không sử dụng	SmoothL1	SmoothL1
Gradient clipping	Không sử dụng	Max norm 10,0	Max norm 10,0

Bảng 3.6: Các tham số riêng của Q-Learning, DQN và Double DQN [7].

Training seed, evaluation seed và cấu hình của từng thí nghiệm được giữ cố định để hỗ trợ tái lập kết quả.

3.5.2 Quy trình huấn luyện và đánh giá

Mỗi thí nghiệm bắt đầu bằng việc khởi tạo môi trường và tác tử theo cấu hình tương ứng. Trong từng episode, tác tử liên tục tương tác với môi trường để cập nhật Q-table hoặc Q-network. Các metric training được ghi lại sau mỗi episode. Cứ 1.000 episode, hệ thống lưu checkpoint và thực hiện một lần evaluation độc lập. Quy trình đầy đủ được tóm tắt trong Hình 3.6.



Hình 3.6: Quy trình training, lưu checkpoint, evaluation và lưu mô hình cuối.

Trong mỗi lần evaluation, ϵ được đặt bằng 0 và tác tử chạy 20 episode với evaluation seed 10042. Tác tử chỉ sử dụng cơ chế chọn action của policy hiện tại. Q-table, replay buffer, optimizer và tham số mạng đều không được cập nhật.

Sau 20.000 episode training, hệ thống lưu mô hình cuối cùng và ghi nhận kết quả của lần evaluation cuối.

Checkpoint và final model phục vụ hai mục đích khác nhau. Checkpoint lưu các thành phần cần thiết để tiếp tục training từ mốc gần nhất. Final model chỉ giữ Q-table đối với Q-Learning hoặc trọng số online network đối với DQN và Double DQN, phù hợp cho evaluation và demo.

3.5.3 Tiêu chí đánh giá

Mỗi lần evaluation gồm 20 episode với $\epsilon = 0$. Thiết lập này giúp kết quả phản ánh trực tiếp policy đã học mà không bị ảnh hưởng bởi hành động thăm dò. Gọi $N = 20$ là số episode trong một lần evaluation. Ở episode thứ i , R_i , T_i và F_i lần lượt biểu thị reward tích lũy, số bước thực hiện và số food đã ăn. Các tiêu chí đánh giá được xây dựng từ ba đại lượng này.

Reward trung bình.

$$\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i. \quad (3.1)$$

Đại lượng $\bar{R}$ phản ánh reward tích lũy trung bình của tác tử. Do reward có thể được cộng từ nhiều sự kiện trong cùng một bước, chỉ số này cần được đọc cùng win rate, completion rate và số bước trung bình thay vì dùng riêng để kết luận về chất lượng policy.

Số bước trung bình.

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i. \quad (3.2)$$

$\bar{T}$ cho biết độ dài trung bình của một episode trước khi tác tử chiến thắng, mất mạng cuối cùng hoặc đạt giới hạn bước. Số bước lớn không đồng nghĩa với policy tốt hơn nên chỉ số này phải được xem cùng win rate và completion rate.

Số food ăn được trung bình.

$$\bar{F} = \frac{1}{N} \sum_{i=1}^N F_i. \quad (3.3)$$

$\bar{F}$ biểu thị lượng food trung bình tác tử thu thập được trong một episode, qua đó cho thấy mức tiến triển của tác tử trên bản đồ.

Completion rate trung bình.

$$\overline{C} = \frac{1}{N} \sum_{i=1}^N \left(\frac{F_i}{62} \times 100\% \right). \quad (3.4)$$

Mỗi episode có tổng cộng 62 food. Vì vậy, $F_i/62$ là tỷ lệ bản đồ được hoàn thành trong episode thứ i , còn $\overline{C}$ là tỷ lệ hoàn thành trung bình của toàn bộ 20 episode.

Win rate.

$$\text{WinRate} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(F_i = 62) \times 100\%. \quad (3.5)$$

Hàm chỉ thị $\mathbf{1}(F_i = 62)$ nhận giá trị 1 khi tác tử ăn hết 62 food và chiến thắng, ngược lại nhận giá trị 0. Win rate trong evaluation được tính độc lập trên 20 episode tại từng mốc. Chỉ số này khác với win rate tích lũy trong lịch sử training.

Reward tốt nhất.

$$R_{\max} = \max_{1 \leq i \leq N} R_i. \quad (3.6)$$

$R_{\max}$ ghi nhận reward tích lũy lớn nhất trong 20 episode, thể hiện kết quả tốt nhất mà policy đạt được ở lần evaluation đó.

Completion rate tốt nhất.

$$C_{\max} = \max_{1 \leq i \leq N} \left(\frac{F_i}{62} \times 100\% \right). \quad (3.7)$$

$C_{\max}$ là tỷ lệ hoàn thành lớn nhất trong 20 episode, cho biết policy đã tiến gần đến việc hoàn thành bản đồ nhất ở mức nào.

Đối với DQN và Double DQN, loss được tính bằng SmoothL1Loss theo phương trình (2.17). Q-Learning không sử dụng chỉ số này vì không huấn luyện mạng nơ-ron. Hệ thống còn thống kê số episode kết thúc do chiến thắng, mất mạng cuối cùng hoặc đạt giới hạn bước. Bảng 3.7 tóm tắt toàn bộ tiêu chí được sử dụng.

Metric	Ý nghĩa và cách tính
Average reward	Reward tích lũy trung bình của 20 episode evaluation, được đọc cùng win rate, completion rate và số bước.
Average steps	Số bước trung bình trước khi episode kết thúc, được đọc cùng win rate và completion rate.
Average food eaten	Số food được ăn trung bình trong 20 episode.
Average completion rate	Trung bình tỷ lệ food đã ăn trên tổng số 62 food của từng episode.
Win rate	Số episode chiến thắng chia cho 20 episode evaluation.
Loss	Trung bình SmoothL1Loss của các update trong một episode training, chỉ có ở DQN và Double DQN.
Best reward	Reward lớn nhất trong 20 episode evaluation.
Best completion rate	Completion rate lớn nhất trong 20 episode evaluation.
Wins, caught, timeout	Số episode kết thúc do thắng, mất mạng cuối cùng hoặc đạt giới hạn bước.

Bảng 3.7: Các tiêu chí được sử dụng để đánh giá kết quả thực nghiệm.

Lần đánh giá cuối sử dụng $\epsilon = 0$ nên phản ánh policy đã học mà không trộn lẫn exploration.

4 KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN

4.1 Kết quả đánh giá cuối

Sau 20.000 episode training, cả chín cấu hình được evaluation độc lập trên 20 episode với $\epsilon = 0$. Tất cả cấu hình sử dụng training seed 42 và evaluation seed 10042. Hai seed được tách riêng, còn evaluation seed được giữ cố định nhằm tăng tính nhất quán và khả năng tái lập kết quả. Bảng 4.1 tổng hợp kết quả của lần evaluation cuối tại episode 20.000 [7].

Thuật toán	LR	Reward Avg	Completion Avg	Win Rate	Steps Avg	Completion Max
Double DQN	0,0001	1247,154	99,76%	95%	148,30	100%
Double DQN	0,0005	1312,231	99,27%	85%	153,60	100%
Double DQN	0,001	1553,422	99,27%	65%	211,65	100%
DQN	0,0001	1125,694	98,55%	75%	157,85	100%
DQN	0,0005	1254,443	99,84%	95%	138,90	100%
DQN	0,001	1148,294	98,71%	75%	161,85	100%
Q-Learning	0,0001	61,960	26,29%	0%	99,25	51,61%
Q-Learning	0,0005	134,079	42,66%	0%	124,05	51,61%
Q-Learning	0,001	165,472	46,29%	0%	134,40	54,84%

Bảng 4.1: Kết quả đánh giá hiệu năng cuối cùng của các thuật toán tại episode 20.000 [7].

Kết quả trong Bảng 4.1 cho thấy ba điểm chính:

- **Khác biệt giữa phương pháp dạng bảng và phương pháp dùng mạng nơ-ron:** Q-Learning không thắng episode nào ở cả ba learning rate, trong khi DQN và Double DQN đạt win rate từ 65% đến 95%. Trong thiết kế thực nghiệm hiện tại, các cấu hình sử dụng Q-network với state 92 chiều đạt kết quả cao hơn Q-Learning sử dụng Q-table với tuple state 17 giá trị. Do đầu vào khác nhau, chênh lệch này chưa thể được quy hoàn toàn cho thuật toán.
- **Hai cấu hình có win rate cao nhất:** DQN với LR = 0,0005 và Double DQN với LR = 0,0001 cùng đạt win rate 95%. DQN hoàn thành các episode evaluation với số bước trung bình thấp hơn, lần lượt là 138,90 và 148,30. Khi win rate và completion rate của hai cấu hình gần tương đương,

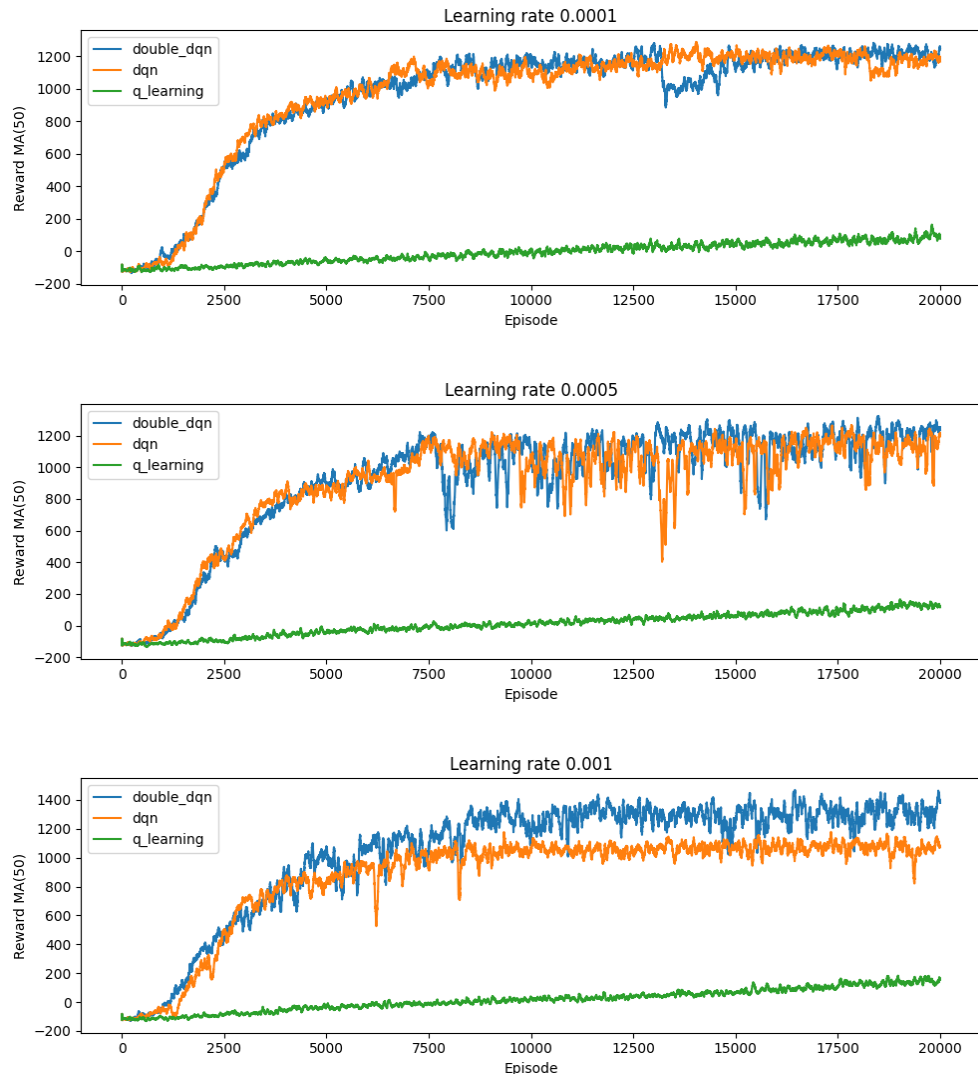
chênh lệch này cho thấy DQN sử dụng ít bước hơn trong lần evaluation cuối. Dữ liệu hiện tại chưa đủ để khẳng định đây là đường đi ngắn nhất.

- **Quan hệ giữa reward và khả năng hoàn thành:** Double DQN với LR = 0,001 đạt reward trung bình cao nhất là 1553,422 nhưng win rate chỉ đạt 65%. Kết quả cho thấy reward tích lũy cao không đồng nghĩa với khả năng hoàn thành màn chơi ổn định. Do log evaluation không tách reward theo từng loại sự kiện, chưa thể xác định chính xác thành phần tạo ra mức chênh lệch này.

4.2 Phân tích quá trình huấn luyện

Các đồ thị trong mục này sử dụng metric ghi nhận sau từng episode training và được làm mượt bằng trung bình trượt 50 episode để thể hiện xu hướng. Các bảng tại mốc 1.000, 5.000, 10.000, 15.000 và 20.000 episode sử dụng kết quả evaluation độc lập trên 20 episode với $\epsilon = 0$ và seed 10042. Hai nguồn dữ liệu được trình bày song song nhằm phản ánh cả diễn biến trong quá trình training và hiệu năng của policy tại từng checkpoint. Các bảng tập trung vào ba cấu hình có kết quả cuối tốt nhất của từng thuật toán: Double DQN với LR = 0,0001, DQN với LR = 0,0005 và Q-Learning với LR = 0,001.

4.2.1 Reward



Hình 4.1: Reward trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.

Reward tích lũy phản ánh tổng tác động của các sự kiện xảy ra trong một episode. Do nhiều thành phần reward có thể được cộng trong cùng một bước, chỉ số này cần được đọc cùng completion rate và win rate. Bảng 4.2 trình bày reward trung bình từ các lần evaluation tại năm mốc training.

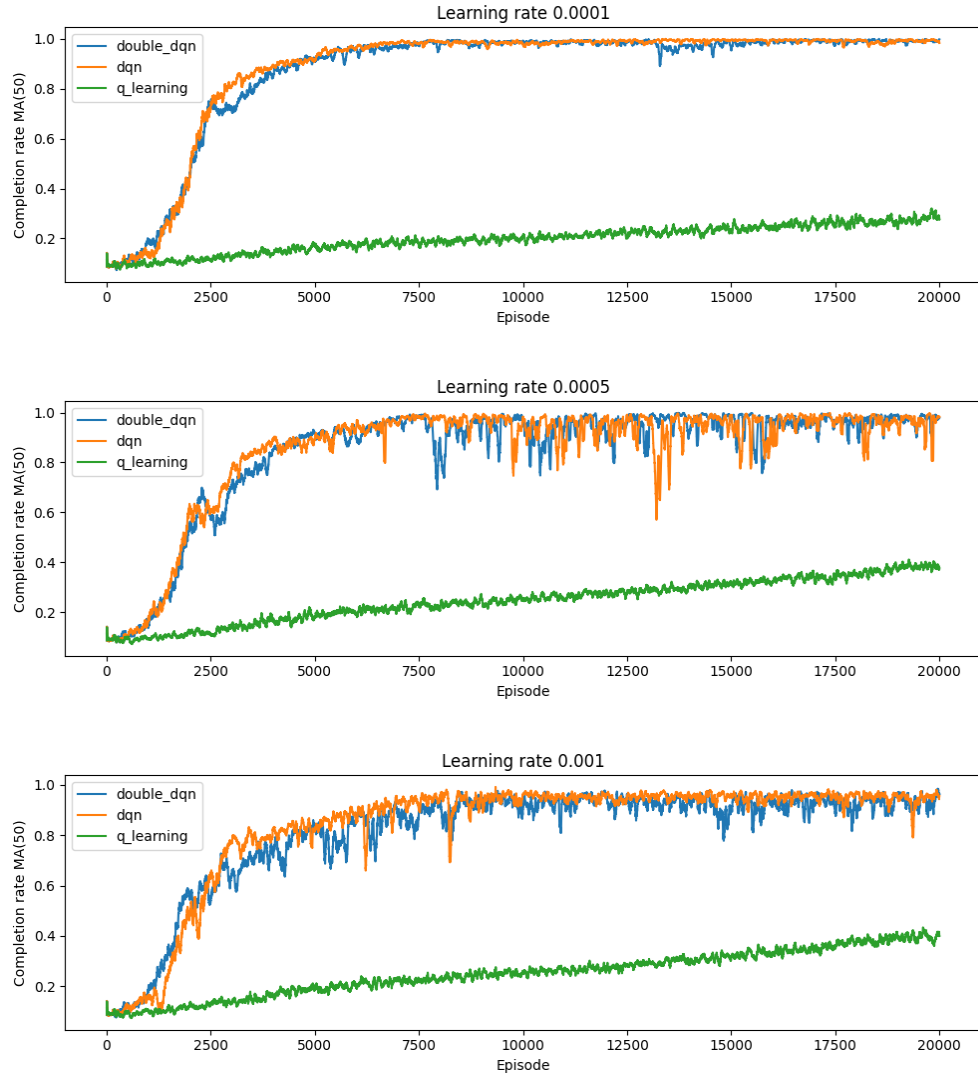
Cấu hình	1.000 episode	5.000 episode	10.000 episode	15.000 episode	20.000 episode
Double DQN (LR = 0,0001)	239,490	946,191	1195,874	1159,812	1247,154
DQN (LR = 0,0005)	302,023	969,347	982,988	1257,292	1254,443
Q-Learning (LR = 0,001)	-155,458	-34,216	16,860	106,130	165,472

Bảng 4.2: Reward trung bình tại các mốc đánh giá.

Hình 4.1 cho thấy reward training của DQN và Double DQN tăng nhanh trong giai đoạn đầu, sau đó dao động quanh mức cao. Tuy nhiên, reward tăng chưa đồng nghĩa với policy đã biết kết thúc màn chơi. Tại episode 5.000, Double DQN và DQN đạt reward evaluation lần lượt là 946,191 và 969,347, đồng thời đều thu thập hơn 95% food. Dù vậy, hai cấu hình chỉ thắng lần lượt 1 và 0 trong 20 episode. Kết quả này cho thấy reward ở giai đoạn đó chủ yếu phản ánh khả năng thu thập phần lớn food và duy trì episode, còn khả năng hoàn thành vẫn chưa ổn định.

Đến episode 15.000, DQN đạt reward 1257,292, completion rate 100%, win rate 100% và chỉ cần trung bình 134,25 bước. Sự thay đổi đồng thời của bốn chỉ số cho thấy policy không chỉ tạo reward cao hơn mà còn chuyển từ việc gần hoàn thành sang hoàn thành ổn định với ít bước hơn. Q-Learning cũng cải thiện reward từ -155,458 lên 165,472, đi kèm completion rate tăng từ 8,39% lên 46,29%. Tuy nhiên, win rate vẫn bằng 0%, nên mức reward tăng của Q-Learning chỉ phản ánh tiến bộ từng phần chứ chưa hình thành policy giải quyết hoàn chỉnh bài toán.

4.2.2 Completion rate



Hình 4.2: Completion rate trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.

Completion rate biểu thị tỷ lệ food tác tử thu thập được trên tổng số 62 food của bản đồ. Bảng 4.3 trình bày completion rate trung bình từ các lần evaluation tại năm mốc training.

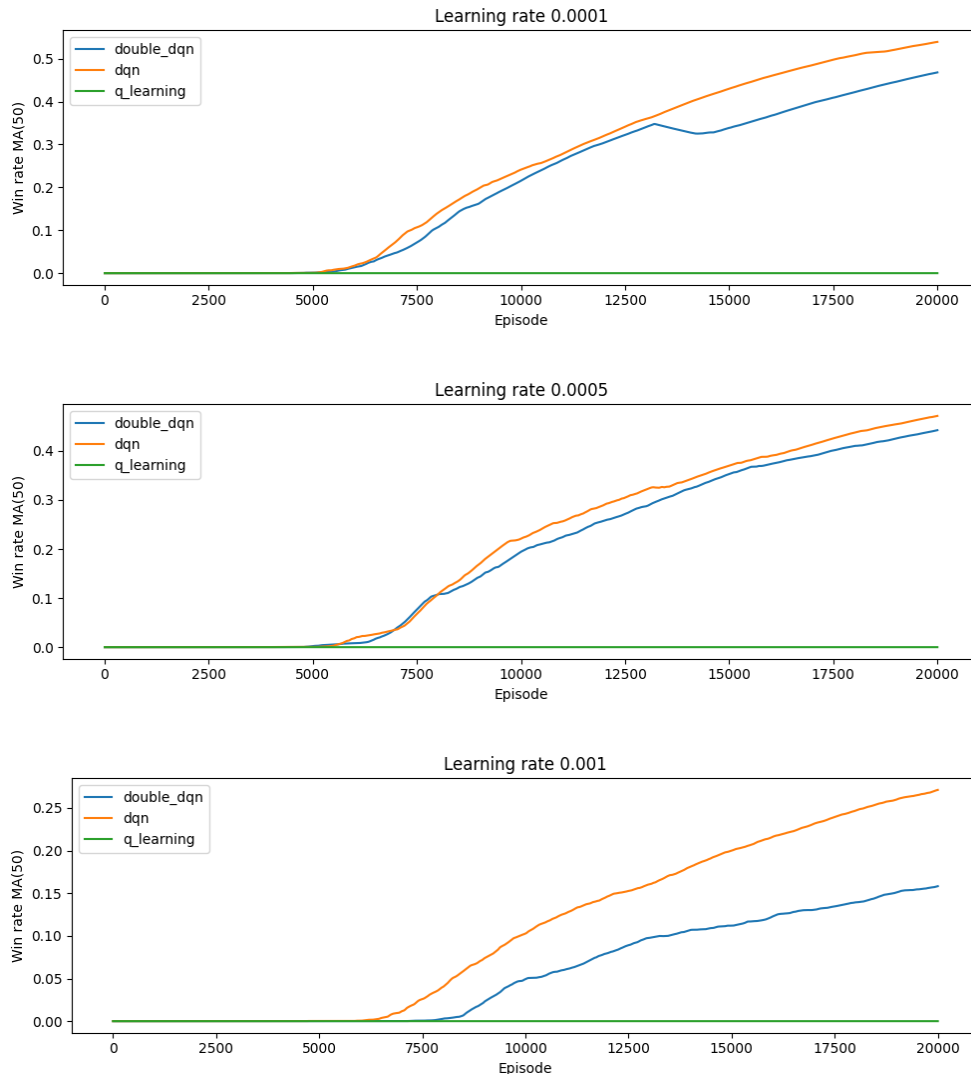
Cấu hình	1.000 episode	5.000 episode	10.000 episode	15.000 episode	20.000 episode
Double DQN (LR = 0,0001)	49,11%	95,32%	99,03%	98,79%	99,76%
DQN (LR = 0,0005)	58,71%	95,56%	91,21%	100,00%	99,84%
Q-Learning (LR = 0,001)	8,39%	21,53%	27,74%	36,61%	46,29%

Bảng 4.3: Completion rate trung bình tại các mốc đánh giá.

Completion rate giúp phân biệt hai mức độ học khác nhau: tìm được phần lớn food và thực sự hoàn thành episode. Tại episode 5.000, Double DQN đạt completion rate 95,32% nhưng chỉ thắng 1/20 episode, còn DQN đạt 95,56% nhưng không thắng episode nào. Như vậy, cả hai policy đã học được cách đi sâu vào mê cung và thu thập gần hết food, nhưng vẫn thường bị bắt trước khi lấy các food cuối cùng.

Khoảng cách giữa completion rate và win rate thu hẹp rõ rệt ở các checkpoint sau. DQN đạt đồng thời completion rate và win rate 100% tại episode 15.000. Double DQN kết thúc training với completion rate 99,76% và 19/20 episode chiến thắng. Ngược lại, Q-Learning tăng đều từ 8,39% lên 46,29% nhưng cả 20 episode evaluation cuối đều kết thúc do bị bắt. Q-Learning vì thế có học được hành vi thu thập food tốt hơn theo thời gian, song mức cải thiện chưa đủ để hoàn thành bản đồ.

4.2.3 Win rate



Hình 4.3: Tỷ lệ thắng tích lũy trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.

Win rate trên đồ thị là tỷ lệ thắng tích lũy từ đầu quá trình training. Trong khi đó, Bảng 4.4 sử dụng win rate của 20 episode evaluation độc lập tại từng checkpoint. Ở cả hai trường hợp, một episode được tính là thắng khi tác tử thu thập đủ 62 food.

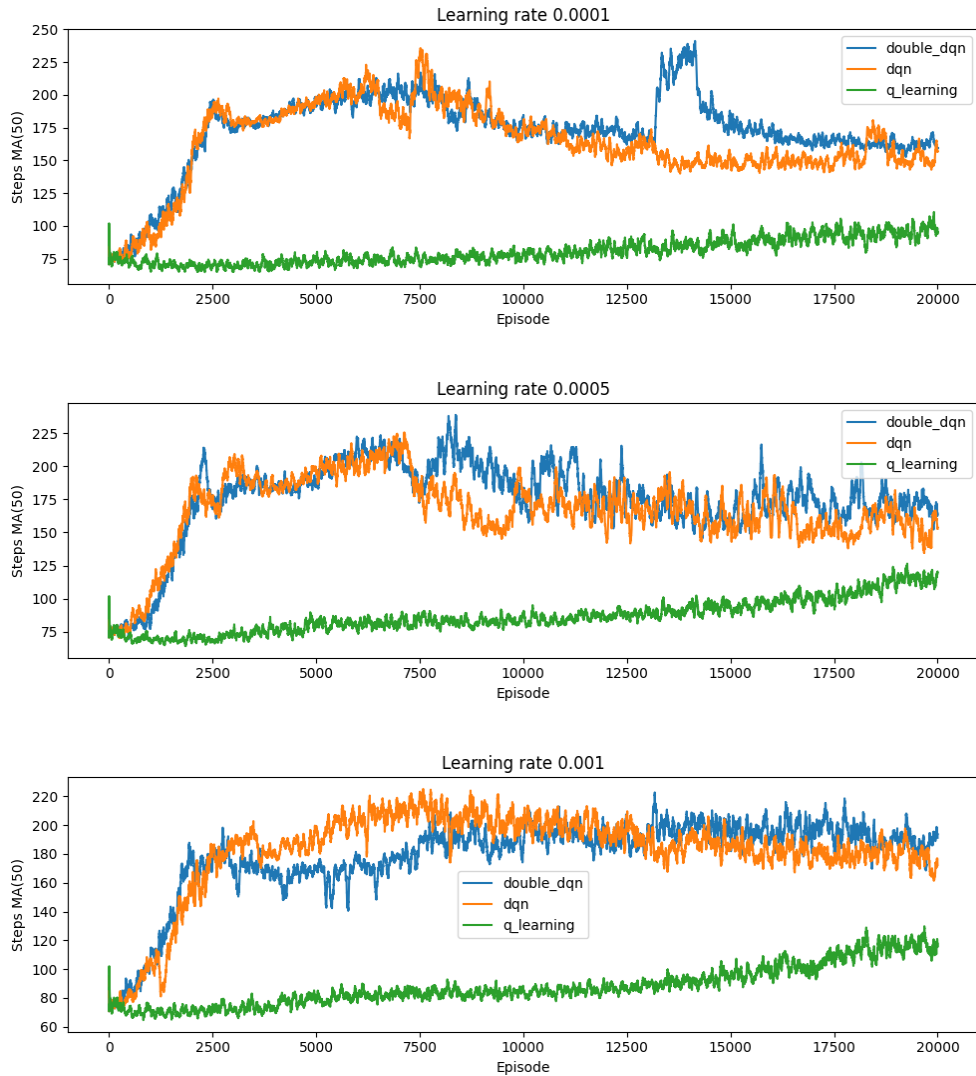
Cấu hình	10.000 episode	11.000 episode	12.000 episode	15.000 episode	20.000 episode
Double DQN (LR = 0,0001)	80%	95%	85%	80%	95%
DQN (LR = 0,0005)	45%	0%	100%	100%	95%
Q-Learning (LR = 0,001)	0%	0%	0%	0%	0%

Bảng 4.4: Win rate tại các mốc đánh giá.

Win rate cho thấy thời điểm policy chuyển từ gần hoàn thành sang hoàn thành thực sự. DQN có win rate 45% tại episode 10.000, thấp hơn mức 80% của Double DQN. Sau đó, DQN biến động mạnh và đạt 100% tại episode 12.000, đồng thời duy trì mức này ở episode 15.000.

Dao động lớn nhất xuất hiện ở DQN tại episode 11.000. Reward giảm từ 982,988 xuống 138,031, completion rate giảm từ 91,21% xuống 38,31% và cả 20 episode đều kết thúc do bị bắt. Ngay tại episode 12.000, policy lại thắng 20/20 episode với completion rate 100%. Sự suy giảm đồng thời trên nhiều metric cho thấy đây là một checkpoint có hiệu năng thấp, không chỉ là biến động riêng của win rate. Dù vậy, một training seed chưa đủ để quy nguyên nhân cho catastrophic forgetting. Double DQN cũng dao động từ 95% ở episode 11.000 xuống 80% ở episode 15.000, nhưng không xuất hiện mức sụt về 0% như DQN.

4.2.4 Steps



Hình 4.4: Số bước trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.

Số bước trung bình phản ánh độ dài của episode trước khi tác tử thắng, mất mạng cuối cùng hoặc đạt giới hạn bước. Vì episode thất bại sớm cũng có thể có ít bước, chỉ số này phải được đọc cùng win rate và completion rate. Bảng 4.5 trình bày số bước trung bình từ các lần evaluation tại năm mốc training.

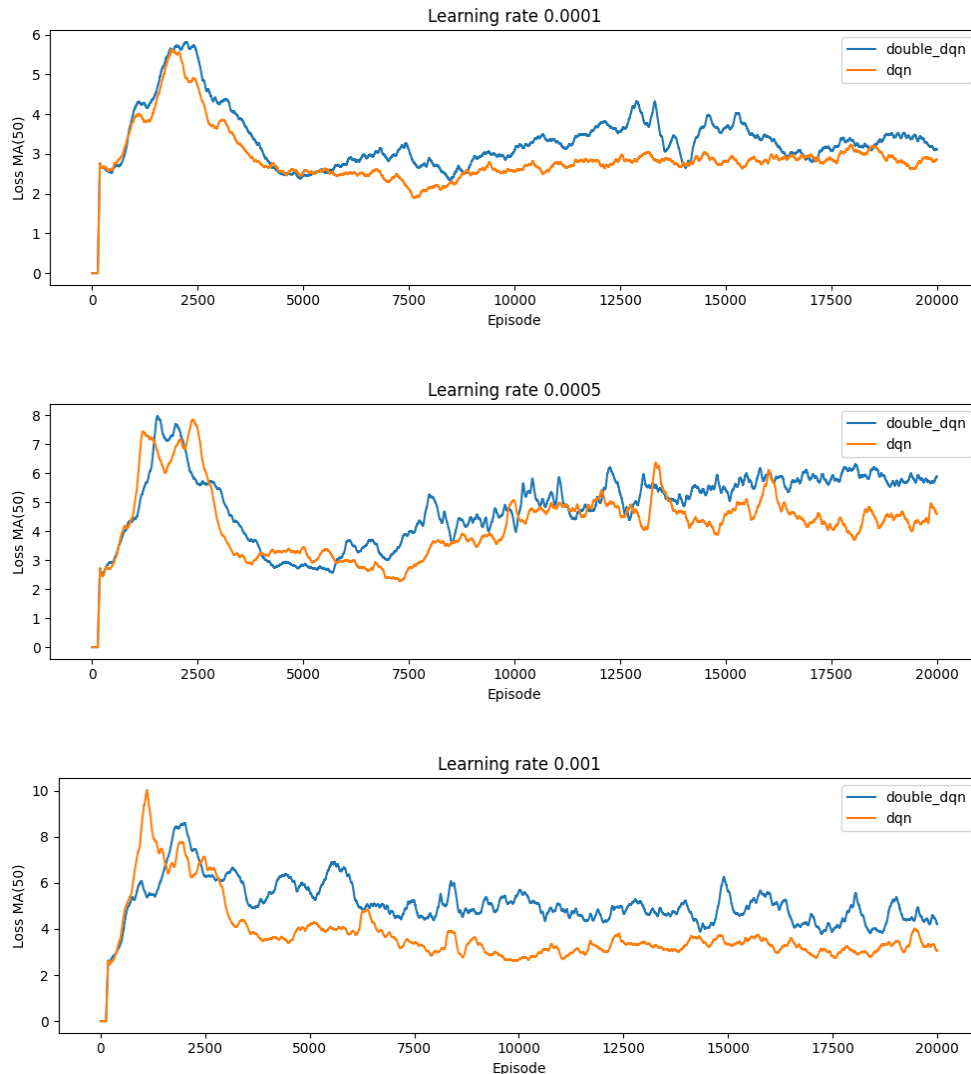
Cấu hình	1.000 episode	5.000 episode	10.000 episode	15.000 episode	20.000 episode
Double DQN (LR = 0,0001)	219,00	235,60	163,25	176,35	148,30
DQN (LR = 0,0005)	210,90	229,95	177,85	134,25	138,90
Q-Learning (LR = 0,001)	66,00	85,20	89,05	99,60	134,40

Bảng 4.5: Số bước trung bình tại các mốc đánh giá.

Số bước chỉ có ý nghĩa khi được đối chiếu với kết quả kết thúc episode. Tại episode 5.000, DQN và Double DQN cần trung bình khoảng 230 bước, đạt completion rate trên 95% nhưng gần như không thắng. Các episode dài ở giai đoạn này cho thấy tác tử đã tồn tại đủ lâu để thu thập phần lớn food, nhưng chưa hoàn thành được phần cuối của bản đồ. Đến episode 15.000, DQN thắng 20/20 episode và số bước giảm còn 134,25. Việc số bước giảm đồng thời với win rate tăng cho thấy policy đã hoàn thành episode hiệu quả hơn, thay vì kết thúc sớm do bị bắt.

Tại lần evaluation cuối, DQN và Double DQN cùng thắng 19/20 episode, với số bước trung bình lần lượt là 138,90 và 148,30. DQN sử dụng ít bước hơn trong cùng điều kiện hoàn thành gần tương đương, nhưng dữ liệu chưa đủ để khẳng định đây là đường đi ngắn nhất. Đối với Q-Learning, số bước tăng từ 66,00 lên 134,40 cùng với completion rate tăng từ 8,39% lên 46,29%. Vì tất cả episode cuối vẫn kết thúc do bị bắt, xu hướng này phản ánh khả năng tồn tại và thu thập food được cải thiện, không phải hiệu quả hoàn thành.

4.2.5 Loss



Hình 4.5: Smooth L1 Loss trong quá trình training, được làm mượt bằng trung bình trượt 50 episode, với các mức learning rate lần lượt từ trên xuống dưới là 0,0001, 0,0005 và 0,001.

Smooth L1 Loss chỉ áp dụng cho DQN và Double DQN. Chỉ số này là trung bình loss của các lần cập nhật mạng trong một episode training. Bảng 4.6 tổng hợp giá trị tại một số mốc của hai cấu hình có win rate cuối cao nhất.

Cấu hình	5.000 episode	10.000 episode	15.000 episode	20.000 episode
DQN (LR = 0,0005)	3,42	4,74	4,60	4,78
Double DQN (LR = 0,0001)	2,43	3,28	3,67	2,86

Bảng 4.6: Loss trung bình trong episode tại các mốc training.

Trước khi replay buffer tích lũy đủ 10.000 environment step, loss bằng 0 vì online network chưa được cập nhật. Khi quá trình tối ưu bắt đầu, loss tăng nhanh rồi chuyển sang trạng thái dao động. Loss không giảm đều về 0 do target thay đổi theo chính policy đang học và phân phối mẫu trong replay buffer cũng liên tục biến đổi.

DQN với LR = 0,0005 có loss 4,60 tại episode 15.000 và 4,78 tại episode 20.000, dù win rate tương ứng là 100% và 95%. Điều này cho thấy loss không cần đạt giá trị nhỏ nhất để policy có hiệu năng cao. Double DQN với LR = 0,0001 có loss thấp hơn tại các mốc được thống kê, nhưng hai cấu hình sử dụng learning rate khác nhau nên không thể quy toàn bộ chênh lệch cho cơ chế Double DQN. Loss vì thế chỉ được dùng để theo dõi quá trình cập nhật mạng, còn chất lượng policy phải được đánh giá bằng reward, completion rate và win rate.

4.3 Ảnh hưởng của learning rate

Learning rate (α) ảnh hưởng khác nhau đến từng thuật toán. Bảng 4.7 sắp xếp kết quả theo thứ tự LR = 0,0001 $\rightarrow$ 0,0005 $\rightarrow$ 0,001 để thể hiện trực tiếp xu hướng của từng chỉ số.

Thuật toán	Số thắng/20	Completion rate (%)	Steps
Q-Learning	0 $\rightarrow$ 0 $\rightarrow$ 0	26,29 $\rightarrow$ 42,66 $\rightarrow$ 46,29	99,25 $\rightarrow$ 124,05 $\rightarrow$ 134,40
DQN	15 $\rightarrow$ 19 $\rightarrow$ 15	98,55 $\rightarrow$ 99,84 $\rightarrow$ 98,71	157,85 $\rightarrow$ 138,90 $\rightarrow$ 161,85
Double DQN	19 $\rightarrow$ 17 $\rightarrow$ 13	99,76 $\rightarrow$ 99,27 $\rightarrow$ 99,27	148,30 $\rightarrow$ 153,60 $\rightarrow$ 211,65

Bảng 4.7: Xu hướng kết quả khi learning rate tăng từ 0,0001 lên 0,001 [7].

Q-Learning. Trong ba cấu hình đã chạy, learning rate tăng đi kèm với reward, completion rate và số bước trung bình cùng tăng. LR = 0,001 đạt reward 165,472 và completion rate 46,29%, cao hơn hai mức còn lại, nhưng cả 20

episode evaluation vẫn kết thúc do bị bắt. Như vậy, learning rate lớn hơn giúp policy đi xa hơn và thu thập nhiều food hơn trong miền đã khảo sát, song chưa làm thay đổi kết quả cuối cùng là win rate 0%. Do learning rate lớn nhất mới chỉ bằng 0,001, thí nghiệm chưa xác định được mức phù hợp nhất cho cập nhật Q-table.

DQN. Hai mức còn lại là $LR = 0,0001$ và $LR = 0,001$ đều thắng 15/20 episode, trong khi $LR = 0,0005$ thắng 19/20 episode. Cấu hình này cũng có completion rate cao nhất là 99,84% và số bước thấp nhất là 138,90. Cả ba chỉ số đều cho thấy $LR = 0,0005$ là cấu hình cân bằng nhất của DQN trong miền đã khảo sát. Tuy nhiên, ba điểm dữ liệu và một training seed chưa đủ để kết luận quan hệ giữa learning rate và hiệu năng có dạng cụ thể.

Double DQN. Khi learning rate tăng, số episode thắng giảm lần lượt từ 19 xuống 17 và 13, trong khi số episode bị bắt tăng từ 1 lên 3 và 7. Completion rate của cả ba cấu hình vẫn trên 99%, cho thấy các episode thất bại thường kết thúc khi tác tử đã thu thập gần hết food. Đồng thời, reward tăng từ 1247,154 lên 1553,422 và số bước tăng từ 148,30 lên 211,65. Vì log không tách reward theo sự kiện, chưa thể xác định nguồn reward tăng thêm. Tuy nhiên, kết quả đủ cho thấy $LR = 0,001$ tạo ra policy có reward cao nhưng mất nhiều bước hơn và bị bắt thường xuyên hơn. Xét mục tiêu hoàn thành màn chơi, $LR = 0,0001$ là cấu hình phù hợp nhất của Double DQN trong ba mức đã thử.

4.4 Tổng hợp kết quả thực nghiệm

Tại episode 20.000, Bảng 4.8 đối chiếu cấu hình có win rate cao nhất của từng thuật toán. Cột kết quả lần lượt ghi số episode thắng, bị bắt và timeout trong 20 episode evaluation.

Cấu hình	Thắng/Bị bắt /Timeout	Reward	Completion rate	Steps
Q-Learning ($LR = 0,001$)	0/20/0	165,472	46,29%	134,40
DQN ($LR = 0,0005$)	19/1/0	1254,443	99,84%	138,90
Double DQN ($LR = 0,0001$)	19/1/0	1247,154	99,76%	148,30

Bảng 4.8: Đối chiếu ba cấu hình đại diện tại episode 20.000 [7].

Q-Learning không thắng episode nào và cả 20 episode đều kết thúc do bị bắt.

Completion rate 46,29% cho thấy tác tử đã học được cách thu thập một phần food, nhưng chưa học được policy hoàn thành bản đồ.

DQN và Double DQN có cùng số episode thắng và cùng số lần bị bắt. DQN có completion rate cao hơn 0,08 điểm phần trăm và sử dụng ít hơn trung bình 9,40 bước. Chênh lệch này ủng hộ việc chọn DQN LR = 0,0005 khi ưu tiên số bước, nhưng một training seed chưa đủ để khẳng định DQN tốt hơn Double DQN về tổng quát.

Reward cũng không thể được dùng làm tiêu chí duy nhất. Double DQN với LR = 0,001 đạt reward cao nhất là 1553,422, nhưng chỉ thắng 13/20 episode, bị bắt 7 lần và cần trung bình 211,65 bước. Vì vậy, DQN LR = 0,0005 và Double DQN LR = 0,0001 là hai cấu hình cân bằng nhất khi xét đồng thời win rate, completion rate và số bước.

4.5 Hạn chế của thực nghiệm

Mỗi cấu hình chỉ được training một lần với seed 42. Vì chưa có các lần chạy độc lập, báo cáo chưa xác định được độ biến động giữa các seed, độ lệch chuẩn hoặc khoảng tin cậy. Những chênh lệch nhỏ giữa các cấu hình vì thế cần được diễn giải thận trọng.

Mỗi lần evaluation gồm 20 episode với seed 10042 nên win rate thay đổi theo bước 5%. Seed cố định tạo điều kiện so sánh nhất quán, nhưng chưa phản ánh hiệu năng trên nhiều chuỗi ngẫu nhiên. Log cũng chỉ lưu reward tổng, chưa tách đóng góp của các sự kiện như ăn ghost, bonus fruit hoặc va tường.

Q-Learning dùng tuple state 17 giá trị, còn DQN và Double DQN dùng vector 92 chiều với nhiều thông tin bổ sung. Chênh lệch kết quả do đó chịu ảnh hưởng đồng thời của thuật toán và cách biểu diễn state. Thiết kế hiện tại chưa tách riêng hai yếu tố này.

Ba learning rate được dùng chung cho cả ba thuật toán, dù chúng tác động lên Q-table và optimizer Adam theo cách khác nhau. Miền khảo sát chưa chắc chứa cấu hình tốt nhất cho từng phương pháp. Kết quả chỉ cho phép so sánh ba mức đã chọn.

Thực nghiệm mới được thực hiện trên bản đồ medium, với ba ma, ba mạng sống và một thiết kế reward cố định. Khả năng khái quát sang điều kiện khác cần được kiểm chứng bằng thí nghiệm bổ sung.

4.6 Hướng dẫn chạy demo

Demo cho phép lựa chọn một trong chín final model đã huấn luyện của Q-Learning, DQN và Double DQN. Trong quá trình chạy, tác tử chỉ sử dụng policy đã lưu để lựa chọn action. Q-table và tham số mạng không được cập nhật. Các lệnh dưới đây được thực hiện trong PowerShell tại thư mục gốc của repository [7].

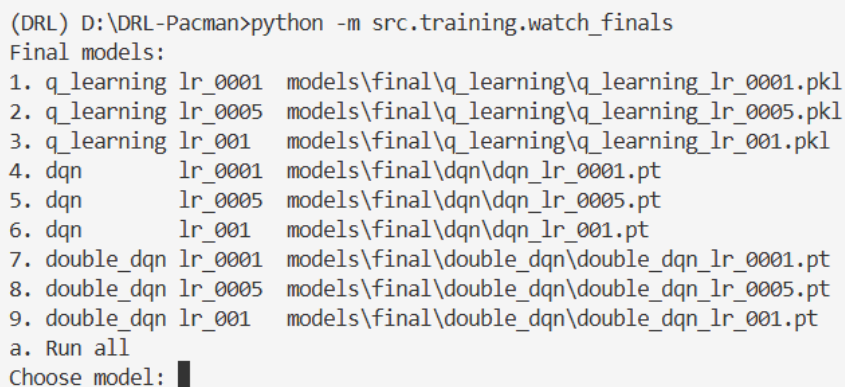
1. **Chuẩn bị môi trường.** Bước này chỉ cần thực hiện trong lần chạy đầu tiên.

```
conda create -n pacman python=3.10 -y
conda activate pacman
pip install -r requirements.txt
```

2. **Mở danh sách lựa chọn model.** Chạy lệnh sau để chương trình quét thư mục models/final và hiển thị các model hiện có.

```
python -m src.training.watch_finals
```

Mỗi dòng trong danh sách gồm số thứ tự, thuật toán, learning rate và đường dẫn tới final model. Người dùng nhập số tương ứng rồi nhấn Enter để chạy model. Chẳng hạn, nhập 5 để chọn DQN với LR = 0,0005, là cấu hình DQN có kết quả cuối tốt nhất trong thực nghiệm. Lựa chọn a dùng để chạy lần lượt toàn bộ model. Giao diện lựa chọn được minh họa trong Hình 4.6.

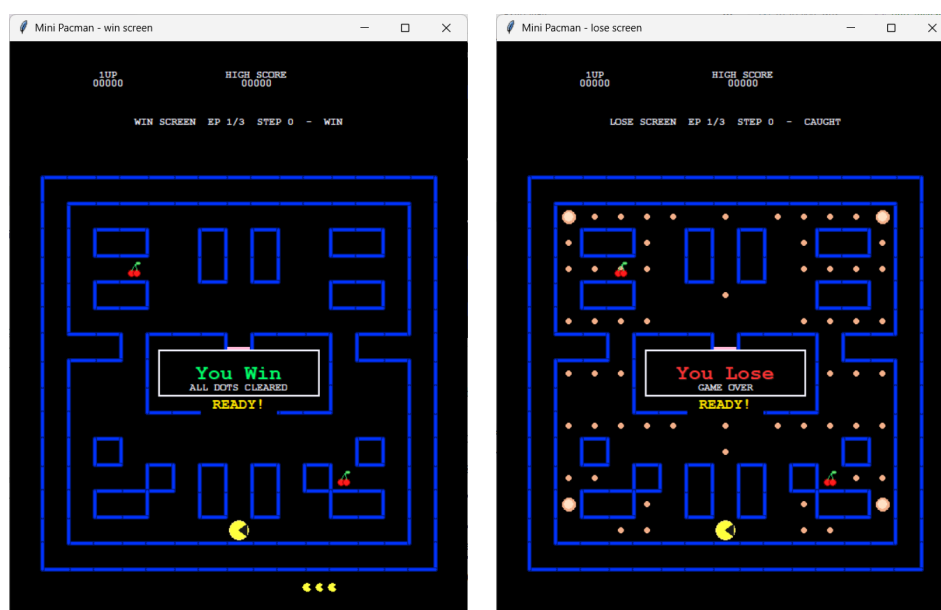


```
(DRL) D:\DRL-Pacman>python -m src.training.watch_finals
Final models:
1. q_learning lr_0001 models\final\q_learning\q_learning_lr_0001.pkl
2. q_learning lr_0005 models\final\q_learning\q_learning_lr_0005.pkl
3. q_learning lr_001 models\final\q_learning\q_learning_lr_001.pkl
4. dqn lr_0001 models\final\dqn\dqn_lr_0001.pt
5. dqn lr_0005 models\final\dqn\dqn_lr_0005.pt
6. dqn lr_001 models\final\dqn\dqn_lr_001.pt
7. double_dqn lr_0001 models\final\double_dqn\double_dqn_lr_0001.pt
8. double_dqn lr_0005 models\final\double_dqn\double_dqn_lr_0005.pt
9. double_dqn lr_001 models\final\double_dqn\double_dqn_lr_001.pt
a. Run all
Choose model: █
```

Hình 4.6: Danh sách lựa chọn final model khi chạy demo.

3. **Quan sát tác tử.** Sau khi model được chọn, cửa sổ Mini Pacman mở và policy tự động điều khiển Pacman. Phần trên của giao diện hiển thị episode hiện tại, số bước, action vừa chọn và sự kiện trong môi trường. Mặc định, chương trình chạy ba episode với độ trễ 500 ms giữa hai bước.

Khi Pacman thu thập đủ 62 food, episode kết thúc với thông báo *You Win*. Nếu Pacman mất mạng cuối cùng hoặc chạm giới hạn số bước, giao diện hiển thị *You Lose*. Hai trạng thái kết thúc được thể hiện trong Hình 4.7.



(a) Pacman hoàn thành bản đồ.

(b) Pacman kết thúc episode do bị bắt.

Hình 4.7: Hai trạng thái kết thúc có thể quan sát trong giao diện demo.

4. **Điều chỉnh hiển thị.** Số episode và tốc độ hiển thị có thể được thay đổi bằng các tham số sau.

```
python -m src.training.watch_finals --episodes 5 --delay-ms 150
```

Trong lệnh trên, `--episodes 5` đặt số episode chạy cho model được chọn, còn `--delay-ms 150` rút ngắn thời gian chờ giữa hai bước xuống 150 ms. Sau khi nhập lệnh, người dùng vẫn chọn model từ danh sách như ở bước 2.

Người dùng có thể đóng cửa sổ Mini Pacman tại bất kỳ thời điểm nào để kết thúc demo hiện tại.

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài đã xây dựng môi trường Mini Pacman, cài đặt Q-Learning, DQN và Double DQN, sau đó thực hiện chín cấu hình thí nghiệm trên bản đồ medium. Kết quả thu được đáp ứng các mục tiêu nghiên cứu trong phạm vi đã xác định ở Chương 1. Những kết luận chính gồm:

1. **Về hệ thống và quy trình thực nghiệm.** Môi trường mô phỏng đầy đủ các thành phần cần thiết của bài toán gồm Pacman, food, power pellet, bonus fruit, ba ma, số mạng và điều kiện kết thúc episode. Repository tổ chức cấu hình bằng YAML, hỗ trợ lưu checkpoint, tiếp tục training, ghi metrics, lưu final model, vẽ biểu đồ và chạy evaluation độc lập với $\epsilon = 0$ [7]. Nhờ đó, chín cấu hình được thực hiện theo cùng một quy trình và có thể đối chiếu bằng cùng nhóm chỉ số.
2. **Về kết quả của ba thuật toán.** Bảng 4.8 cho thấy sự khác biệt rõ ràng giữa ba cấu hình đại diện tại episode 20.000.
 - Q-Learning với LR = 0,001 không thắng episode nào, cả 20 episode đều kết thúc do bị bắt và completion rate đạt 46,29%. Kết quả cho thấy Q-table đã tạo ra tiến bộ về khả năng thu thập food, nhưng chưa hình thành policy hoàn thành bản đồ trong giới hạn training của đề tài.
 - DQN với LR = 0,0005 thắng 19/20 episode, đạt completion rate 99,84% và cần trung bình 138,90 bước. Đây là cấu hình DQN cân bằng nhất trong ba learning rate đã thử.
 - Double DQN với LR = 0,0001 cũng thắng 19/20 episode, đạt completion rate 99,76% và cần trung bình 148,30 bước. Kết quả gần tương đương DQN về khả năng hoàn thành, nhưng số bước trung bình cao hơn 9,40.

Trong thiết kế hiện tại, hai cấu hình sử dụng Q-network đạt kết quả cao hơn Q-Learning. Tuy nhiên, Q-Learning dùng tuple state 17 giá trị, còn DQN

và Double DQN dùng vector 92 chiều. Vì vậy, chênh lệch quan sát được chịu ảnh hưởng đồng thời của thuật toán và cách biểu diễn state, không thể được quy hoàn toàn cho riêng thuật toán.

3. **Về learning rate và cách đọc metrics.** Bảng 4.7 cho thấy learning rate phù hợp phụ thuộc vào từng phương pháp. DQN đạt kết quả tốt nhất tại mức giữa $LR = 0,0005$. Với Double DQN, số episode thắng giảm từ 19 xuống 17 và 13 khi learning rate tăng. Q-Learning cải thiện completion rate khi learning rate tăng trong miền khảo sát, nhưng win rate vẫn bằng 0%.

Reward cần được đọc cùng win rate, completion rate và steps. Double DQN với $LR = 0,001$ đạt reward cao nhất là 1553,422, nhưng chỉ thắng 13/20 episode và cần trung bình 211,65 bước. Tương tự, sự suy giảm của DQN tại episode 11.000 xuất hiện đồng thời trên nhiều metrics, nhưng một training seed chưa đủ để xác định nguyên nhân là overestimation bias hay catastrophic forgetting.

Nhìn chung, đề tài đã hoàn thành việc xây dựng môi trường, cài đặt ba thuật toán và tổ chức quy trình so sánh bằng dữ liệu định lượng. Các kết quả chỉ đại diện cho chín cấu hình, một training seed, một evaluation seed và bản đồ medium. Do đó, chúng không được xem là kết luận chung về hiệu quả của các thuật toán trong mọi bài toán học tăng cường.

5.2 Hướng phát triển

Các hướng phát triển được đề xuất trực tiếp từ những hạn chế của thiết kế thực nghiệm hiện tại:

1. Tăng độ tin cậy của kết quả.

- Training mỗi cấu hình với nhiều seed độc lập, sau đó báo cáo giá trị trung bình, độ lệch chuẩn và khoảng tin cậy.
- Tăng số episode evaluation và sử dụng nhiều evaluation seed để giảm ảnh hưởng của một chuỗi ngẫu nhiên cố định.

- Khảo sát learning rate riêng cho từng thuật toán thay vì dùng chung ba mức cho cả Q-table và optimizer Adam.

2. Tách ảnh hưởng của thuật toán và biểu diễn state.

- Xây dựng thí nghiệm ablation để so sánh các thuật toán trên cùng tập đặc trưng đầu vào.
- Thử các biến thể state có và không có thông tin về food gần nhất, ma gần nhất, action hợp lệ và action nguy hiểm.
- Khi đã có đường cơ sở công bằng, có thể thử đầu vào ảnh kết hợp mạng nơ-ron tích chập để đánh giá khả năng tự trích xuất đặc trưng.

3. Mở rộng metrics và thiết kế reward.

- Tách reward theo từng sự kiện như ăn food, ăn ghost, lấy bonus fruit, mất mạng và va tường.
- Ghi thêm số lần đổi hướng, chọn action nguy hiểm và thời gian ở trạng thái frightened để định lượng hành vi trong demo.
- Thử các cấu hình reward khác nhau và đánh giá bằng cả win rate, completion rate lẫn steps thay vì chỉ dựa trên reward tổng.

4. Mở rộng môi trường và thuật toán.

- Thử nghiệm trên nhiều bản đồ, số lượng ma và mức độ truy đuổi khác nhau để kiểm tra khả năng khái quát của policy.
- Nghiên cứu Dueling DQN và Prioritized Experience Replay trên cùng quy trình evaluation hiện tại.
- Thử các phương pháp Actor-Critic hoặc Proximal Policy Optimization để mở rộng so sánh ngoài nhóm thuật toán dựa trên giá trị.

TÀI LIỆU THAM KHẢO

- [1] R. S. Sutton and A. G. Barto, [*Reinforcement Learning: An Introduction*](#), 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, “[Q-Learning](#),” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “[Playing Atari with Deep Reinforcement Learning](#),” *arXiv preprint arXiv:1312.5602*, 2013.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “[Human-Level Control Through Deep Reinforcement Learning](#),” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] H. van Hasselt, A. Guez, and D. Silver, “[Deep Reinforcement Learning with Double Q-Learning](#),” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016.
- [6] University of California, Berkeley, “[CS 188 Project 3: Reinforcement Learning](#),” Department of Electrical Engineering and Computer Sciences, Pacman Artificial Intelligence Projects, 2024.
- [7] Đặng Anh Tuyền, “[DRL-Pacman: Q-Learning, DQN and Double DQN on Mini Pacman](#),” GitHub repository, 2026.
- [8] TS. Trần Đình Tân, “Nhập môn trí tuệ nhân tạo,” Trường Công nghệ Thông tin Phenikaa.